ДРАФТ_

ФСП федерация спортивного программирования России

СОПРОВОДИТЕЛЬНАЯ ДОКУМЕНТАЦИЯ

Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника Федерации спортивного программирования: обратная механика подбора, адаптивное тестирование, интеграция с ФСП ID.

Хакатон ФСП · 10.10.2026
Репозиторий: README.md · API: /api/docs · Демо: docker compose up

СОДЕРЖАНИЕ

1. Архитектура решения
2. Механика тестирования кандидатов
3. Механика подбора и категоризации
4. Собственная процедура валидации и её результаты
5. Схема интеграции с ФСП ID и реестром достижений ФСП
6. Описание API
7. Используемые библиотеки и компоненты
8. Сборка, развёртывание и локальный запуск
9. Безопасность и 152-ФЗ
10. Антифрод, интеграция с ATS и развитие после MVP
11. Соответствие решениям сессии вопросов и ответов

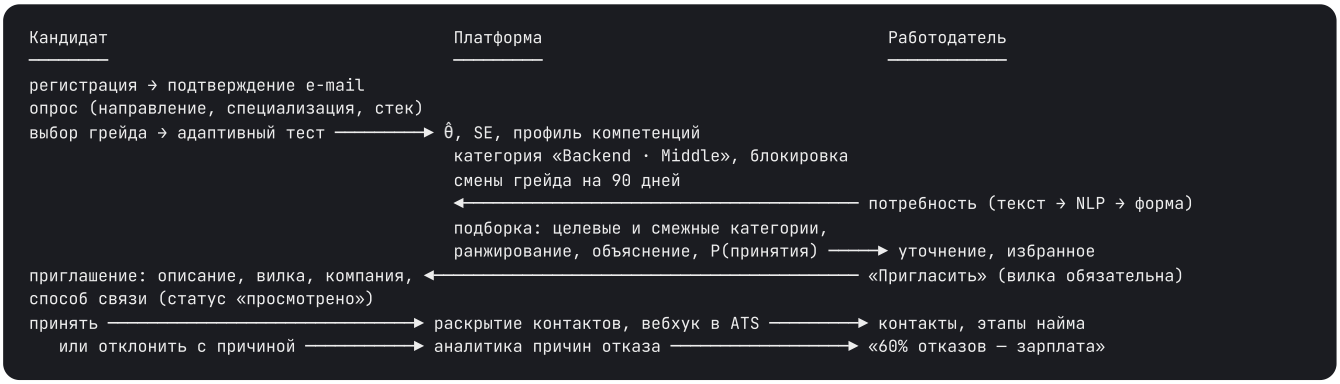
1. АРХИТЕКТУРА РЕШЕНИЯ

1.1. Функциональная архитектура

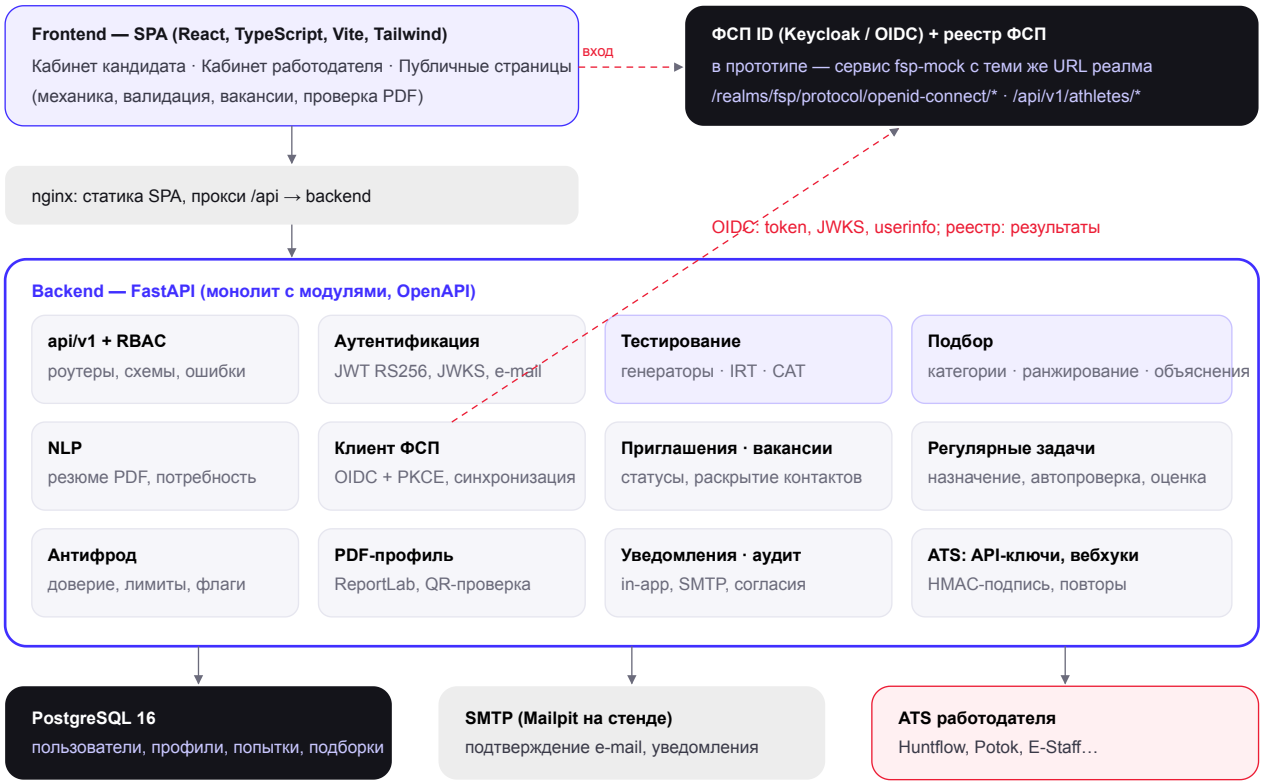
Платформа соединяет напрямую двух участников — кандидата и работодателя. Сторонние вакансии не агрегируются.

Роль	Функции
Кандидат	регистрация по e-mail с подтверждением или вход через ФСП ID; профиль вручную или из PDF-резюме (автораспознавание ФИО, контактов, стека, стажа, грейда, ролей, софт-скиллов); **опрос** по направлению (отрасли ИТ) и специализации; выбор предполагаемого грейда; **адаптивный тест** → категория «специализация × грейд»; история опросов, попыток и смен грейда; пересдача уровнем ниже/выше с ограничением частоты; **входящие приглашения** с вилкой и способом связи (принять / отклонить с причиной / пожаловаться); вакансии и отклики; регулярные задачи от работодателей; приватность, журнал согласий 152-ФЗ, выгрузка и удаление данных; связь с ФСП ID и синхронизация достижений; стандартизированный PDF-профиль с QR-проверкой; **переписка** с работодателем после принятия приглашения и **отзыв доступа к контактам**; регистрация с 15 лет (15–17 — с согласием законного представителя); **рост и зарплаты**: зарплатный радар категории (обезличенно) и навыки, которые просят на следующем грейде
Работодатель	профиль компании и индекс доверия; **потребность** («кого ищем») формой или текстом с NLP-разбором; **подборка**: рекомендованные категории + ранжированные кандидаты с объяснением; уточнение подборки без её потери, избранное и скрытые; карта категорий и поиск по банку кандидатов (специализация, грейд, стек, подтверждённость, ФСП, формат, город, ожидания); **приглашение** конкретному кандидату или группе (вилка обязательна) и отслеживание статусов и этапов найма; аналитика (конверсия, причины отказов); вакансии, отклики, рекомендованные кандидаты под вакансию; план теста под потребность/вакансию; регулярные задачи и оценка решений; API-ключи и вебхуки для ATS; **переписка** с кандидатом после принятия приглашения; кандидаты с неподтверждённым грейдом видны со статусом и ниже подтверждённых; **сравнение 2–3 кандидатов** «почему A выше B»
Публичная часть	лендинг, описание механики подбора, интерактивная демонстрация механики тестирования (генерация вариантов, симулятор CAT), отчёт валидации, проверка подлинности PDF-профиля, список вакансий
Администратор	сводка, жалобы, подозрительные попытки, онлайн-калибровка банка (API)

Сквозной сценарий (обратная механика)



1.2. Компонентная архитектура



Компонент	Технологии	Ответственность
frontend	React 18, TypeScript, Vite, Tailwind CSS, TanStack Query, Recharts	SPA для обеих ролей и публичных страниц; адаптивная вёрстка; в Docker — nginx со статикой и прокси `/api`
backend	Python 3.13, FastAPI, SQLAlchemy 2, Pydantic 2	REST API (OpenAPI), RBAC, бизнес-логика; монолит с явным разделением слоёв и слабо связанными модулями
PostgreSQL	PostgreSQL 16	реляционное хранилище, JSONB для гибких структур (профиль компетенций, объяснения)
fsp-mock	FastAPI, PyJWT	демо-стенд ФСП ID с URL и форматом токенов Keycloak + API реестра достижений; заменяется настоящими сервисами ФСП без изменения кода платформы
Mailpit	—	SMTP-ловушка с веб-интерфейсом для демонстрации подтверждения e-mail

Модули backend

```
app/
  api/v1/        HTTP-слой: auth, candidate, assessment, employer, public (+ATS, admin). Зависимости RBAC: require_role,
current_candidate
  core/          конфигурация, БД, безопасность (Argon2id, JWT RS256, JWKS), ошибки, rate limiting
  models/        ORM-модели (users, candidate_profiles, test_attempts/items, hiring_needs, shortlist_entries, invitations, ...)
  schemas/       Pydantic-схемы запросов (валидация, описание полей для OpenAPI)
  catalog/       справочники: направления, специализации, грейды, компетенции, онтология ~130 навыков, дисциплины ФСП
  testing/       bank/ (226 шаблонов-генераторов), irt.py (3PL, EAP, person-fit), cat.py (адаптивный алгоритм), blueprint.py,
service.py
  matching/      scoring.py (формула и объяснения), service.py (подборки, поиск, карточки), strength.py, text.py (TF-IDF)
  nlp/           resume_parser.py (PDF → поля), need_parser.py (текст потребности → запрос)
  fsp/           client.py (OIDC + PKCE, реестр), scoring.py (оценка достижений)
  services/      invitations, tasks, trust (антифрод), webhooks (ATS), pdf, common (почта, уведомления, аудит, согласия)
  seed/          синтетические данные
  validation/    Монте-Карло тестирования, оценка подбора, калибровка, графики
```

Модули общаются через функции сервисного слоя; HTTP-слой не содержит бизнес-логики. Механики тестирования и подбора (`testing/cat.py`, `matching/scoring.py`) — чистые функции над dataclass-представлениями, поэтому одна и та же логика используется в API, в генерации синтетических данных и в процедуре валидации.

1.3. Модель данных (основные сущности)

Таблица	Назначение
`users`	учётные записи (роль, e-mail, Argon2id-хеш пароля, подтверждение e-mail, `fsp_sub` — связь с ФСП ID)
`candidate_profiles`	три слоя: **самоописание** (резюме, заявленный грейд и навыки, возрастная группа 18+ / 15–17), **верифицированный слой** (грейд, θ̂ ± SE, профиль компетенций, подтверждённые навыки, ФСП-оценка), **служебный** (сила профиля, перцентиль в категории); настройки приватности и видимости
`candidate_categories`	дополнительные категории кандидата (до двух): свой грейд, результат теста, блокировка и подтверждённые навыки в каждой
`survey_responses`, `grade_changes`	история опросов и изменений грейда
`test_attempts`, `test_items`	попытки и выданные экземпляры заданий: текст варианта, серверный ключ ответа, ответ, время, события фокуса/вставки, IRT-параметры
`item_stats`	экспозиция и калиброванные параметры семейств заданий, флаг дрейфа
`fsp_achievements`	результаты соревнований из реестра ФСП (подписанные записи)
`companies`	профиль работодателя, индекс доверия и его факторы
`hiring_needs`, `shortlist_entries`	потребности и сохранённые подборки (оценка, ранг, объяснение, пометки работодателя)
`invitations`	приглашения: описание, вилка (gross), способ связи, статус, этап найма, причина отказа, объяснение, отзыв доступа к контактам
`messages`	переписка кандидата и работодателя внутри принятого приглашения (без обмена контактами)
`vacancies`, `applications`	вакансии (в том числе синхронизированные из ATS по `external_id`) и самостоятельные отклики
`employer_tasks`, `task_assignments`	регулярные задачи и решения кандидатов
`consents`, `audit_log`, `notifications`, `refresh_tokens`, `oidc_states`	152-ФЗ, аудит, уведомления, сессии, состояния OIDC
`reports`, `api_keys`, `webhooks`, `webhook_deliveries`	антифрод и интеграция с ATS

1.4. Фирменный стиль ФСП

Интерфейс, письма, PDF-профиль кандидата, демо-страница ФСП ID и эта документация оформлены по брендбуку ФСП («Руководство по использованию фирменного стиля»):

Элемент брендбука	Как применён
Цвета логотипа #402FFF, #EC1D35, #EDEDED; градации серого #1B1C21 · #2D2E37 · #3A3C46 · #9B9C9E · #D9D9D9	токены темы (`tailwind.config.js`): основной цвет действий — #402FFF, акцент и курсор знака — #EC1D35, тёмные поверхности — графит #1B1C21 / #2D2E37 / #3A3C46
Digital-акцент #0A006B (сайт, соцсети, баннеры)	градиент тёмных полос: главная, страница входа, обложка документации
Основной шрифт JetBrains Mono, акцентный TDAText	заголовки, подписи, цифры — JetBrains Mono; словесный знак «ДРАФТ_» и крупные числа — TDAText; длинный текст — Inter (основной шрифт флагманских проектов ФСП). Все три — локально, без внешних CDN

Элемент брендбука	Как применён
Логотип ФСП (полноцвет, для тёмного и светлого фона)	официальные файлы брендбука без изменений: сокращённый вариант — в шапке рядом с «Драфтом» (как у флагманских проектов: собственный знак + логотип Федерации), полный — в подвале, на странице входа и обложке документации; знак-звезда — на кнопке «Войти через ФСП ID» и в иконке сайта
Гибридные паттерны (символы кода)	полоса паттерна в цветах #402FFF / #EC1D35 / #EDEDED: низ первого экрана, подвал, меню кабинета, страница входа, обложка документации
Цветовой код дисциплин ФСП (фиолетовый — продуктовое, синий — алгоритмическое, бирюзовый — БАС, жёлтый — робототехника, зелёный — ИБ)	значки и метки достижений ФСП в профиле кандидата
Голос бренда: «_» перед пунктами, моноширинные заголовки прописными	подзаголовки разделов, панель входа, заголовки глав документации

Логотип Министерства спорта из комплекта не используется: прототип не является официальным материалом министерства.

1.5. Нефункциональные свойства

- **Производительность.** Ранжирование подборки по ~700 кандидатам — десятки миллисекунд; выдача следующего задания CAT — < 50 мс. Две копии uvicorn-воркеров в контейнере; ресурсы стенда < 1 ГБ RAM (замер под нагрузкой — ниже).
- **Кэш дорогих чтений.** Поиск по банку кандидатов, карта категорий и карточки кандидатов кэшируются в памяти воркера; ключ включает версию данных (время последнего изменения профилей), поэтому изменения видны сразу. Воркеров uvicorn — по числу ядер (до 4).
- **Масштабирование.** Backend без состояния (JWT), горизонтально масштабируется за балансировщиком; инициализация БД защищена advisory-lock PostgreSQL. Тяжёлые фоновые операции (вебхуки) — в фоне ответа; для продакшена выносятся в очередь (см. п. 10).
- **Платформенность.** Docker-образы на Debian slim/alpine, запуск на любой ОС с Docker; адаптивная вёрстка для смартфонов и планшетов.
- **Качество кода.** Сложные решения прокомментированы в коде (IRT, CAT, генераторы, формула подбора); 278 автотестов бэкенда, 14 тестов компонентов и 7 браузерных сценариев фронтенда.
- **Без LLM и без GPU.** Большие языковые модели **не используются** ни в обязательном пути, ни опционально: задания порождаются детерминированными генераторами с вычисляемым ответом (ключи проверены исполнением кода, п. 2.1), разбор резюме и вакансий, категоризация и подбор — словари, правила, TF-IDF и интерпретируемые формулы на CPU. Поэтому персональные данные не передаются внешним сервисам (в том числе трансгранично), нет затрат на API, а результат воспроизводим. Организаторы допускают облачные LLM, в том числе зарубежные, с предпочтением открытых моделей разумного размера; если позже понадобится LLM (например, объяснение подборки текстом), предполагается открытая модель, развёрнутая у оператора платформы, и передача ей только обезличенного профиля (категория, навыки, без ФИО и контактов) — так не возникает трансграничной передачи персональных данных по 152-ФЗ.
- **Ресурсы стенда проверки.** Весь стек (PostgreSQL, backend с 4 воркерами, frontend, fsp-mock, Mailpit) занимает ≈ 1 ГБ RAM под нагрузкой. Замер в Docker на 5 ядрах и 4 ГБ, сценарии подборки, поиска, карточек и кабинетов без пауз между запросами: **20 одновременных пользователей — 498 запросов/с, p95 ≤ 0,13 с; 50 — 432 запроса/с, p95 ≤ 0,30 с; 0 ошибок.** До кэша и масштабирования воркеров было 150 запросов/с и p95 до 0,73 с. С большим запасом укладывается в ориентир жюри (≈ 8 CPU, 32–64 ГБ, без GPU).

2. МЕХАНИКА ТЕСТИРОВАНИЯ КАНДИДАТОВ

Требования Т3: категория кандидата определяется обязательным путём — опрос, самостоятельный выбор грейда и тест на заявленный грейд; единый набор заданий не годится (утекает), генерация уникальных заданий нейросетью не гарантирует сопоставимой сложности и объективности. Нужен подход, устойчивый к обеим проблемам.

Решение «Драфта»: банк шаблонов-генераторов + теория ответов на задания (IRT) + адаптивный тест (CAT).

Проблема	Как решается
Задания и ответы расходятся по рынку	В банке хранятся не задания, а **семейства-генераторы**. Каждый кандидат получает свои варианты: другие данные, код, таблицы, наборы утверждений, порядок ответов. **Ответ не хранится, а вычисляется** генератором (исполнением SQL, симуляцией алгоритма, пересчётом формулы)
Уникальные задания несопоставимы по сложности	Варианты семейства **изоморфны** — та же когнитивная операция с другими данными. Параметры трудности калибруются на уровне семейства (IRT 3PL), итоговая оценка — на общей шкале θ
Нужна объективная оценка разных кандидатов	Все кандидаты специализации измеряются на одной шкале способности θ; решение о грейде — по вероятности превышения порога, а не по «проценту верных»
Тест должен быть коротким	Адаптивный выбор: каждое следующее задание максимально информативно около текущей оценки, тест заканчивается, когда решение о грейде надёжно → в среднем ~15 заданий (8–22) вместо 30–40 фиксированных

2.1. Банк шаблонов-генераторов

226 семейств (148 параметрических генераторов и 78 комбинаторных пулов) для 8 специализаций и 26 компетенций, 4 уровня трудности (стажёр — senior). Форматы: ввод ответа (118), одиночный выбор (56), множественный выбор (52).

Три способа порождать варианты

1. **Параметрическая генерация с вычисляемым ответом** (большинство): данные генерируются случайно, ответ вычисляется кодом. * SQL: случайные таблицы (`employees` , `orders` , `customers` с NULL), запрос с JOIN / GROUP BY / оконной функцией; эталонный ответ — **результат исполнения запроса во встроенной SQLite**. * Алгоритмы: случайный граф → порядок BFS/DFS; ключи → индекс в хеш-таблице с пробированием; массив → состояние после прохода пузырька; сетка с препятствиями → число путей (ДП); рюкзак; Дейкстра; LRU-промахи. * Трассировка кода: сгенерированная программа → что она выведет (с учётом семантики языка). * Расчётные задачи: precision/recall/F1/ROC-AUC по сгенерированным данным, шаг градиентного спуска, энтропия/Джини, k-means, z-тест A/B, перцентили, retention и воронки, бюджет ошибок SLO, HPA Kubernetes, CIDR и разбиение подсетей, правила межсетевого экрана, flex-grow, em/rem, специфичность CSS, порядок event loop JS/Dart, React state updates, Diffie–Hellman, RSA, XOR, энтропия пароля.
2. **Комбинаторные пулы утверждений** для понятийных знаний: из пула верных и неверных фактов собирается набор («отметьте все верные»); число комбинаций растёт комбинаторно — «заучить ответ» можно, только выучив предмет.
3. **Одна структура — 9 языков.** Универсальные семейства (циклы, рекурсия, словари, целочисленное деление, ссылочная семантика, try/finally) рендерятся в синтаксис языка кандидата: Python, JavaScript/TypeScript, Java, Go, C#, Kotlin, PHP, Swift, Dart. Ответ вычисляется **с учётом семантики языка**: `-17 // 5` в Python = −4, а `-17 / 5` в Java = −3; `%` в Dart — евклидов остаток; массивы в Swift и PHP копируются при присваивании, в Java/JS/Python — нет. Плюс языковые «ловушки»: event loop и `var/let` в JS, Integer cache в Java, `defer` и `append` в Go, отложенный LINQ в C#, `foreach` по ссылке в PHP.

Пример: семейство `sql.left_join` у двух кандидатов (фрагменты):

Кандидат А	Кандидат Б
`customers` : 6 строк (Москва, Казань…), `orders` : 8 строк	`customers` : 5 строк, `orders` : 9 строк
`SELECT COUNT(*) FROM customers c LEFT JOIN orders o … WHERE o.id IS NULL;` → **1**	тот же запрос по другим данным → **2**

Ключ ответа хранится только на сервере (`test_items.answer_key`) и никогда не отправляется клиенту.

Ключи ответов проверены исполнением

Генератор вычисляет ответ сам, моделируя семантику языка. Ошибка в такой модели означала бы неверный ключ и несправедливую оценку, поэтому ключи сверяются **независимым исполнением** (`app/validation/code_exec.py`): фрагмент, показанный кандидату, дописывается до программы (класс `Main` , `package main` , импорты), запускается настоящим компилятором, и вывод сравнивается с ключом. SQL-запросы исполняются в SQLite по сгенерированным таблицам.

Что исполняется	Чем	Вариантов	Совпало с ключом
Python (13 семейств, включая pandas)	CPython 3.12	78	78
JavaScript (11)	Node.js 20	66	66
C# (11)	.NET 8	66	66
PHP (10)	PHP 8.3	60	60
Swift (9)	Swift 6	54	54
Dart (9)	Dart 3	54	54
Kotlin (9)	Kotlin 2.0	54	54
Java (8)	JDK 21	48	48
Go (8)	Go 1.23	48	48
SQL (13 семейств)	SQLite	390	390
Итого		**918**	**918**

Шесть случайных зёрен на каждое семейство кода и тридцать на SQL. Расхождений не найдено.

Автотест `tests/test_code_exec.py` исполняет SQL и языки, установленные на машине. Все 9 языков сразу проверяет команда `python -m app.validation.code_exec --docker` , которая запускает официальные Docker-образы компиляторов; CI (`.github/workflows/ci.yml`) ставит компиляторы и прогоняет проверку при каждой загрузке.

Состав банка по специализациям (семейств на компетенцию плана)

Специализация	План теста (доли компетенций)	Семейств
Backend	язык 25%, БД и SQL 20%, алгоритмы 15%, HTTP/API 15%, архитектура 15%, Git/Docker/Linux 10%	90
Frontend	язык JS/TS 25%, CSS 15%, браузер и DOM 15%, фреймворк 15%, HTTP 15%, алгоритмы 15%	71
Mobile	язык 25%, мобильные платформы 30%, архитектура 15%, алгоритмы 20%, HTTP 10%	65
Data Science / ML	ML 25%, метрики и математика ML 25%, Python 15%, статистика 15%, SQL 10%, алгоритмы 10%	87
Аналитика данных	SQL 30%, статистика и A/B 25%, продуктовые метрики 20%, Python/pandas 15%, визуализация 10%	66
DevOps / SRE	Linux 20%, сети 20%, контейнеры 20%, CI/CD 15%, SRE 15%, скрипты 10%	60

Специализация	План теста (доли компетенций)	Семейств
QA	тест-дизайн 30%, HTTP/API 20%, SQL 15%, процессы QA 15%, автотесты 10%, нагрузочное 10%	70
Информационная безопасность	веб-безопасность 25%, криптография 20%, сети 20%, расследование 20%, Linux 15%	35

Компетенция «язык программирования» содержит 8–13 семейств на каждый языковой трек. Покрытие «компетенция × уровень» проверено скриптом: у каждой специализации есть задания всех четырёх уровней.

2.2. Модель IRT (3PL)

Вероятность верного ответа кандидата со способностью θ на задание семейства:

```
P(θ) = c + (1 - c) / (1 + exp(-a · (θ - b)))
```

- **b — трудность**: номинальный уровень семейства → b: стажёр −1.6, junior −0.5, middle +0.55, senior +1.55 (плюс поправка семейства);
- **a — дискриминативность**: априорно по формату — ввод ответа 1.8, множественный выбор 1.6, одиночный 1.3 (для пулов −0.1);
- **c — угадывание**: одиночный выбор 0.2, множественный 0.05, ввод 0.02.

Априорные значения уточняются **онлайн-калибровкой** (п. 2.6). Шкала грейдов (пороги θ): стажёр ≥ −2, junior ≥ −1, middle ≥ 0, senior ≥ +1.

2.3. Адаптивный алгоритм (CAT)

1. **Старт**: $\hat{\theta}_0$ — центр заявленного грейда (априорное N(центр, 1)).
2. **Выбор задания**: * *контентная балансировка* — берётся компетенция с наибольшим дефицитом относительно плана теста; * среди семейств этой компетенции (подходящих языку/фреймворку кандидата, не показанных в попытке и, по возможности, в прошлых попытках кандидата) считается информация Фишера $I(\hat{\theta})$; если $\hat{\theta}$ ближе 0.4 к порогу грейда — в точке порога: решение «выше/ниже» сильнее всего уточняют задания, трудность которых совпадает с границей; * *контроль экспозиции (randomesque)*: случайный выбор из топ-4 по информации с весом, уменьшающимся для семейств, которые показываются чаще целевых 25% попыток; * из семейства генерируется новый вариант (зерно = хеш(seed попытки, позиция)).
3. **Оценка** после каждого ответа — EAP (апостериорное среднее на сетке из 161 точки): устойчива при малом числе ответов.
4. **Остановка — классификационная**: не меньше 8 заданий и каждая компетенция плана затронута (до 14-го задания); стоп, как только по **каждому** порогу грейда апостериорная вероятность «выше» или «ниже» достигла 85%. Если решение спорное, тест продолжается — до 22 заданий. Очевидные случаи завершаются за 8–12 заданий, пограничные получают больше. По Монте-Карло это правило точнее прежнего «стоп при SE ≤ 0.35» (точный грейд 83% против 81%, повтор даёт тот же грейд в 75% против 73%) и при этом короче в среднем (14.6 против 15.4 задания) — см. раздел 4.2.
5. **Итог**: $\hat{\theta}$ пересчитывается с *популяционным* априорным N(0, 1.2) — заявленный грейд не влияет на решение.

Время: на каждое задание свой лимит (90–270 с) по серверным часам, на попытку — 90 минут (типичная попытка — 35–45 минут).

2.4. Решение о грейде и правила пересдачи

Ситуация	Правило
Грейд подтверждён	P(θ ≥ порог заявленного грейда) ≥ 0.5

Ситуация	Правило
«Справился уверенно»	P(θ ≥ порог следующего грейда) ≥ 0.8 → кандидату предлагается **отдельный тест на грейд выше** — сразу, без ожидания 90 дней и вне лимита попыток (действует 14 дней, один раз). **Автоматического повышения нет**: грейд меняется, только если пройден этот тест
Не подтвердил	**Грейд не понижается принудительно.** Система подсказывает уровень, который тест подтверждает (наибольший с P ≥ 0.5); тест на уровень ниже доступен **сразу**
Сработали сигналы нечестности	грейд не присваивается до **контрольной формы** (6–10 свежих заданий, строже таймер), доступной сразу и вне лимита (п. 2.6)
Шаг вниз после неудачи	**не расходует попытку** из лимита и не запускает 90-дневную блокировку: подтвердив уровень ниже, кандидат может повторить неудавшийся уровень через обычные 14 дней
Повтор того же или более высокого уровня после неудачи	через 14 дней
Смена подтверждённого грейда	не чаще одного раза в **90 дней**, на одну ступень за раз
Лимит	не более 4 попыток за 30 дней (шаг вниз и тест на повышение по предложению не считаются)
Смена специализации	только после окончания блокировки; категория сбрасывается до нового теста

Обоснование. Грейд присваивает только тест по категории; ни вакансия, ни достижения ФСП, ни резюме грейд не меняют (резюме даёт лишь подсказку при выборе). Кандидат сам выбирает грейд, тест его подтверждает или нет. Скакать по грейдам нельзя: смена — раз в 90 дней и на одну ступень. Но честная неудача не наказывается: шаг вниз бесплатен и не блокирует повторную попытку, а сильному кандидату не нужно ждать 90 дней — он сразу получает отдельный тест на уровень выше.

Все интервалы задаются переменными окружения (`GRADE_CHANGE_COOLDOWN_DAYS`, `RETAKE_SAME_GRADE_COOLDOWN_DAYS`, `MAX_ATTEMPTS_PER_30_DAYS`).

Профиль компетенций и подтверждённые навыки

По каждой компетенции считается своя оценка θ_c с **байесовским сжатием** к общему уровню (априорное $N(\hat{\theta}, 0.8)$) — при 2–3 заданиях оценка темы не «скачет». Результат — радар компетенций для кандидата и работодателя. Навыки компетенции (например, «Базы данных и SQL» → SQL) считаются **подтверждёнными тестом**, если по компетенции ≥ 2 заданий и θ_c не ниже порога грейда − 0.3. Подтверждение навыка участвует в ранжировании (п. 3).

2.5. Почему механика устойчива к распространению заданий

1. **Уже показанный вариант больше никому не выдаётся.** Перед выдачей сервер сверяет хеш содержания варианта (условие, код, таблицы и варианты ответа без учёта их порядка) со всеми выданными ранее и при совпадении генерирует новый (`service.fresh_variant`). Поэтому опубликованный «ответ» всегда относится к чужим данным. Даже без этой проверки два случайных варианта совпадают редко (0.87%, медиана доли уникальных — 91%), но для семейств с небольшим числом вариантов именно проверка свежести делает утечку бесполезной.
2. **Утечка «формы» теста маловероятна.** Формы двух кандидатов одной специализации пересекаются лишь на ~25% *семейств*, а не вариантов; при чистом выборе по максимуму информации было бы 33% и самое популярное семейство показывалось бы в 100% попыток (у нас — ≈ 49% при узкой популяции и 29% в смешанной).
3. **Атака проверена симуляцией**: Junior знает ответы на все варианты, показанные предыдущим 40 кандидатам. Без проверки свежести он отвечает «из утечки» на 9% заданий и проходит Middle в 18% случаев; в «Драфте» — **0% заданий из утечки и 5% прохождений**, как у Junior без всякой утечки (7%). У статичного банка с утечкой 60% ответов — 97%.
4. **Утечку «способа решения» видно по данным.** Если семейство внезапно стало легче, онлайн-калибровка показывает дрейф трудности, и семейство помечается к ревизии (в симуляции найдено 7 из 8 «утёкших»

семейств).

5. **Выучить способ решения = выучить навык.** Знание, что `NOT IN` с `NULL` в подзапросе возвращает пустое множество, и есть компетенция, которую проверяет задание.

2.6. Контроль честности и онлайн-калибровка

- Ключи и проверка — только на сервере; время ответа — по серверным часам; превышение лимита + 20 с — ответ не засчитывается.
- Клиент фиксирует уходы со вкладки и вставки текста; сервер — подозрительно быстрые верные ответы на сложные задания.
- Индекс согласованности ответов **lz** (Drasgow, 1985) выявляет аномальные паттерны («провалил лёгкие, решил трудные»).
- **Контрольная форма.** Если попытка прошла порог, но lz < −2.5 или сработали два поведенческих сигнала, грейд не присваивается и профиль не меняется: кандидат подтверждает результат короткой формой (6–10 свежих заданий у порога, таймер строже на 30%), доступной сразу и вне лимита попыток. В симуляции у честных кандидатов это случается в 0.6% случаев. Честное ограничение: живую подсказку на половине заданий lz распознаёт лишь в ~5% попыток — в коротком адаптивном тесте мало данных; против неё нужен прокторинг (на сессии — «плюс, не обязательство»). Против **утечки** работает гарантия свежих вариантов (п. 2.5).
- `item_stats` копит экспозицию и долю верных. Калибровка (`POST /api/v1/admin/recalibrate`) оценивает трудность b методом максимального правдоподобия при известных θ попыток, сжимает к априорному значению $b = (n \cdot \hat{b} + 20 \cdot b_0)/(n + 20)$ и ставит флаг дрейфа, если $\hat{b} < b_0 - 0.6$. В симуляции ошибка трудности снижается с 0.30 до 0.19.

2.7. Как система формирует тест под описание вакансии

Т3 просит показать, как по описанию вакансии формируются задания. Для потребности или вакансии (`GET /employer/needs/{id}/test-plan` , `GET /employer/vacancies/{id}/test-plan` , кнопка «Как тестировали кандидатов»):

1. навыки вакансии нормализуются по онтологии (`postgres` → PostgreSQL) и сопоставляются компетенциям теста (PostgreSQL → «Базы данных», 0.85; Kafka → «Архитектура», 0.6; Docker → «Git, Docker, Linux»);
2. по навыкам определяется трек (язык, фреймворк);
3. доли базового плана специализации смещаются к покрытым компетенциям: $w'_c = w_c \cdot (1 + 1.2 \cdot \text{доля навыков вакансии в } c)$;
4. показываются план, число семейств на компетенцию и **примеры сгенерированных вариантов** — у каждого кандидата свои.

Навыки, которые тест не проверяет напрямую (например, Figma), учитываются через ФСП, регулярные задачи и резюме.

2.8. Регулярные короткие задания

Задание **адресное**, как в Т3: раз в 7 дней система предлагает кандидату задачу работодателя его категории (приоритет — компании, которые сейчас ищут людей этой категории; затем — наименее загруженные задачи, чтобы одну задачу не решала вся категория и ответы не расходились). Кандидат решает задачу или описывает подход.

Логика оценки — двухступенчатая:

1. **Автоматическая предварительная оценка** по скрытой рубрике работодателя: доля раскрытых ключевых пунктов (нечёткое сравнение) с поправкой на полноту ответа. Работодатель сразу видит, какие пункты не раскрыты.
2. **Проверка на списывание**: если решение почти совпадает с уже отправленным ответом другого кандидата на ту же задачу (сходство ≥ 90% по набору слов, порядок и мелкие правки не скрывают совпадение), работодатель видит пометку «возможное списывание» с кодом второго кандидата.

3. **Итоговая оценка работодателя 1–5** — именно она влияет на профиль. Байесовское среднее (одна случайная «пятёрка» не перевешивает) и свежесть активности входят в компоненту «Актуальность» ранжирования и в силу профиля; навыки задачи с оценкой ≥ 4 становятся доказательством «подтверждён задачей» (сила 0.8).

Работодатель видит автора сильного решения и может сразу его пригласить. Задания под конкретную вакансию — отдельный необязательный блок того же механизма; на грейд регулярные задания не влияют, грейд присваивает только тест.

3. МЕХАНИКА ПОДБОРА И КАТЕГОРИЗАЦИИ

3.1. Принцип: инициатива у работодателя

На досках вакансий кандидат откликается вслепую, а работодатель вручную фильтрует сотни резюме. В «Драфте» — наоборот (ориентир — модель LHH):

1. кандидат **один раз** подтверждает уровень и попадает в категорию;
2. работодатель описывает **потребность** и получает подборку из подходящих категорий с объяснением;
3. работодатель **сам приглашает конкретного человека**: описание, вилка (обязательна), компания, способ связи;
4. кандидат видит условия **до начала общения** и принимает или отклоняет приглашение с причиной;
5. контакты кандидата раскрываются **только после принятия** (или после самостоятельного отклика).

Самостоятельный отклик на вакансию сохранён как дополнительный сценарий для кандидатов, которые пока не получили приглашений.

3.2. Категоризация

Все зарплаты на платформе — **в месяц, до вычета НДФЛ (gross)**; подпись «gross» стоит у каждой суммы в интерфейсе, письмах и API. Если в тексте вакансии вилка указана «на руки», разбор переводит её в gross по ставке НДФЛ 13% и сообщает об этом.

Категория = специализация × грейд, где оба значения получены из обязательного пути (опрос → выбор грейда → тест), а не из резюме. Справочник:

- направления (отрасли ИТ): разработка ПО, данные и ИИ, инфраструктура, качество ПО, кибербезопасность;
- 8 специализаций: Backend, Frontend, Mobile, Data Science / ML, аналитика данных, DevOps / SRE, QA, информационная безопасность;
- 4 грейда: стажёр, junior, middle, senior (порог по шкале теста и типичный стаж);
- онтология ~130 навыков с синонимами и связями (используется для нормализации резюме, потребностей и проверки навыков).

Опрос дополнительно фиксирует основной язык/фреймворк (трек заданий), стек, стаж, роль в команде (исполнитель, ментор, тимлид, техлид), отраслевой опыт (финтех, ритейл, госсектор…) и формат работы — это используется в подборе и объяснении. Грейд — это **уровень навыков, подтверждённый тестом**; стаж остаётся атрибутом профиля и подсказкой при выборе грейда.

Несколько категорий у одного кандидата. Кроме основной категории кандидат может добавить до двух дополнительных (например, backend-разработчик, который работает и как DevOps). Грейд в каждой подтверждается только своим тестом: дополнительная категория сначала видна со статусом «не подтверждён» и ниже подтверждённых, как и основная. Тест проходится по основной категории, поэтому дополнительную можно сделать основной — поля категорий меняются местами, и у каждой сохраняются свой грейд, результат теста, блокировка смены грейда на 90 дней и паузы на пересдачу. Работодатель находит кандидата во всех его категориях: в подборке кандидат участвует лучшей из подходящих запросу категорий, в поиске — той, что совпала с фильтром; в карточке видно, по какой категории он найден, и список остальных («также: DevOps · Junior»). Навык, подтверждённый тестом в любой категории, считается подтверждённым. Карта категорий учитывает дополнительные категории в своих ячейках. (`app/services/categories.py`, `POST /candidate/categories`, `POST /candidate/categories/{spec}/main`)

Внутри категории кандидаты ранжируются по **силе подтверждённого профиля**:

```
strength = 100 · (0.55·T + 0.20·F + 0.15·A + 0.10·P)
  T — положение θ̂ внутри полосы грейда (результат теста),
  F — сила достижений ФСП (0, если истории нет — штрафа нет, бонуса нет),
  A — актуальность: свежесть активности, решённые регулярные задачи и их оценки,
  P — полнота профиля.
```

Для каждой категории считается перцентиль силы — в выдаче: «сильнее 86% кандидатов категории».

3.3. Подборка по потребности: два этапа

Этап 1 — отбор по категориям (recall)

- **целевые категории**: специализация потребности × выбранные грейды;
- **смежный грейд** (±1 ступень) — с понижающим коэффициентом 0.92 и пометкой («на ступень ниже, близок к повышению»);
- **смежная специализация** (backend ↔ devops/qa, ml ↔ аналитика, frontend ↔ mobile…) — коэффициент 0.85.

Выдача упорядочена по категориям: сначала целевая, затем смежные — так выдача формируется на основании категорий, а не текстового совпадения. В подборку не попадают скрытые профили и кандидаты со статусом «не ищу».

Грейд не подтверждён. Кандидат, который прошёл опрос и заявил грейд, но ещё не подтвердил его тестом (или не прошёл тест), **не скрывается**, как рекомендовали постановщики: он показывается в категории заявленного грейда со статусом «грейд не подтверждён тестом» и **после всех подтверждённых** кандидатов. Компонента уровня L для него равна 0, итоговая оценка умножается на 0.6, в объяснении — предупреждение. Работодатель может включить фильтр «только с подтверждённым грейдом», на карте категорий такие кандидаты показаны отдельным числом. Так кандидат не выпадает из оборота, а выдача по-прежнему строится на категориях, подтверждённых тестом.

Этап 2 — ранжирование (precision) с объяснимостью

Интерпретируемая взвешенная сумма шести компонент, каждая в [0, 1]:

Компонента	Вес (экспертный → подобранный)	Как считается
L подтверждённый уровень	0.30 → **0.39**	целевая категория: 0.55 + 0.45·(положение θ в полосе грейда); смежный грейд ниже: 0.2 + 0.35·положение; выше: 0.5
S навыки	0.30 → **0.20**	покрытие обязательных (80%) и желательных (20%) навыков с учётом **силы доказательства**
F достижения ФСП	0.20 → **0.12** × (0.2… 1.0)	оценка достижений с учётом **актуальности дисциплины для запроса** (профильная 1.0, смежная 0.8, непрофильная 0.6; стек решения совпал с навыками запроса — профильная, п. 5.3); долю (20–100% от максимума) задаёт работодатель ползунком «важность ФСП»
C условия	0.15 → **0.15**	0.7·соответствие ожиданий вилке + 0.3·формат и город
A актуальность	0.08 → **0.05**	свежесть активности, регулярные задачи
D контекст	0.07 → **0.08**	отраслевой опыт + TF-IDF-близость описания потребности и профиля

Веса нормируются к 1. Затем применяются **гейт** и множители:

```
score = 100 · Σ wᵢ·компонентаᵢ × (g + (1 - g)·покрытие_обязательных),  g = 0.26 (было 0.50)
        × 0.62, если ожидания выше вилки более чем на 10%   (было 0.85)
        × 0.68, если формат работы несовместим              (было 0.85)
        × 0.90 / 0.88 для смежного грейда / специализации    (было 0.92 / 0.85)
        × 0.60, если грейд заявлен, но не подтверждён тестом
```

Как подобраны веса. Экспертные значения были стартовой точкой. Затем веса и множители подобраны случайным поиском с покоординатным уточнением на **отдельных** наборах потребностей (`python -m app.validation.tune_matching`, результат — `app/matching/tuned_params.json`), а качество измеряется на других, независимых наборах (п. 4): nDCG@10 0.772 → 0.793, P@5 0.828 → 0.848. Подбор идёт **в продуктовых рамках**: навыки ≥ 20%, ФСП ≥ 10%, уровень 20–40%, текст резюме ≤ 12%. Без рамок поиск увеличивал вес текста резюме и обнулял подтверждённые навыки и ФСП — это подгонка под синтетику, противоречащая Т3 (выдача по подтверждённому профилю, ФСП поднимает кандидата). Экспертные веса доступны для

сравнения (`EXPERT_PARAMS`), а одна функция `combine()` считает оценку в продакшене, валидации и подстройке.

Гейт по обязательным навыкам не даёт высокому грейду и хорошим условиям вытянуть наверх человека без нужного стека.

Сила доказательства навыка

Источник	Вес	Когда
Тест	1.0 × доверие	навык заявлен (или это язык/фреймворк из опроса) и подтверждён сильным результатом по соответствующей компетенции; смежные навыки — с коэффициентом (PostgreSQL через «Базы данных» — 0.85, Kafka через «Архитектуру» — 0.6)
ФСП	0.9	навык использован на соревновании (стек результата в реестре ФСП) или прямо проверяется дисциплиной (CTF → веб-безопасность)
Регулярная задача	0.8	навык задачи работодателя, решённой на 4–5
Резюме	0.5	навык только заявлен
Смежный навык	0.3–0.35	есть связанный навык (MySQL при запросе PostgreSQL) или сильная смежная компетенция без заявления навыка

Объяснение выдачи

Каждая компонента возвращает человекочитаемые причины и предупреждения. В карточке кандидата работодатель видит:

- вклад каждой компоненты (полосы и «+29 / +26 / +2.8…»);
- навыки запроса с типом доказательства: **тест**, **ФСП**, **задача**, **резюме**, **смежный**, **нет**;
- причины («Категория Backend · Middle подтверждена адаптивным тестом, 86-й перцентиль», «Подтверждены тестом: Python, SQL», «ФСП: Призёр — Чемпионат СЗФО по продуктовому программированию 2026», «Ожидания 250–300 тыс. ₽ укладываются в вилку»);
- предупреждения («Нет подтверждения обязательных навыков: Kafka», «Ожидания выше вилки на 15%», «Смежная категория…»);
- **вероятность принятия приглашения** (логистическая модель: статус поиска, соответствие вилки, формат, свежесть активности, число висящих приглашений) — помогает выбрать, кому писать, и реалистично назначить вилку.

Сравнение кандидатов «почему A выше B»

В подборке можно выбрать 2–3 кандидатов и открыть сравнение (`GET /employer/needs/{id}/compare`): рядом — итоговая оценка, вклад каждой из шести компонент, навыки запроса с типом доказательства, ожидания относительно вилки, ФСП и вероятность принятия. Для соседей по рейтингу система объясняет разрыв: какие компоненты дали первому больше баллов, в чём сильнее второй и какие множители сработали («ожидания выше бюджета (×0.62)», «покрыто 67% обязательных навыков»). Сравнение считается той же формулой, что и подборка.

Рекомендованные категории

Над списком — карточки категорий: целевые и смежные, число кандидатов, сколько с достижениями ФСП, доля кандидатов, чьи ожидания укладываются в вилку, медианные ожидания («медианные ожидания 440 тыс. ₽ выше вашей вилки»). Клик по карточке уточняет подборку до этой категории.

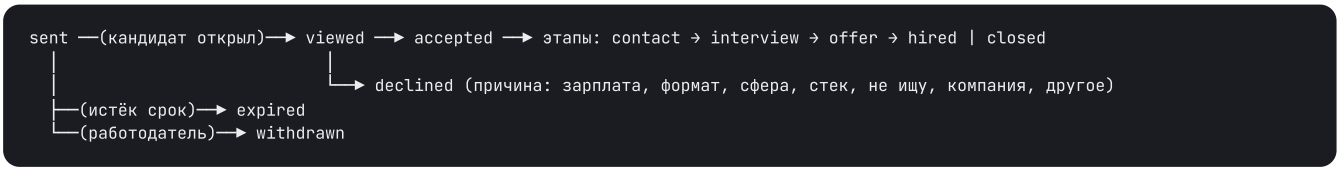
3.4. Уточнение без потери подборки

Подборка сохраняется на сервере (`shortlist_entries`: ранг, оценка, объяснение, пометка). Фильтры (грейд, «обязательно владеет», «только подтверждённые», ФСП, «в моей вилке», формат, минимальное соответствие, отношение к категории) применяются **поверх** сохранённой подборки — счётчик показывает «показано 12 из 60, исходная подборка сохранена», сброс уточнений возвращает всё. Пометки «избранное» и

«скрыт» сохраняются при пересчёте подборки («Обновить подборку»), а кандидат, выпавший из категории, остаётся в избранном.

3.5. Приглашение и его жизненный цикл

Приглашение содержит описание предложения, вилку «от–до» в рублях (обязательно), название компании, способ связи, формат, срок ответа и (необязательно) ссылку на потребность или вакансию.



- Обе стороны видят актуальный статус и этап своих записей; каждое событие — уведомление (в приложении и по e-mail).
- При принятии контакты раскрываются работодателю, событие пишется в аудит и отправляется вебхуком в ATS.
- Причины отказов агрегируются в аналитике работодателя («40% отказов — зарплата ниже ожиданий»).
- Массовое приглашение — до 50 кандидатов одним действием, с лимитами по индексу доверия (п. 10).

3.6. Правила видимости

Данные	До принятия / отклика	После
Категория, θ-перцентиль, профиль компетенций, подтверждённые навыки	видны	видны
Достижения ФСП, результаты задач, ожидания по зарплате	видны, если кандидат не скрыл	видны
Имя	по умолчанию **код кандидата** (снижение предвзятости), кандидат может открыть	видно
Места работы из опыта	видны; текущий работодатель скрыт по умолчанию (чтобы его не узнал нынешний наниматель), кандидат может открыть	видны
E-mail, телефон, Telegram, GitHub	**скрыты**	видны работодателю, которому кандидат ответил

Кандидат видит только свои приглашения и отклики; работодатель — только свои потребности, приглашения, вакансии и задачи (проверено автотестами).

ТЗ закрывает до принятия приглашения только контактные данные; организаторы подтвердили, что ФИО и места работы показывать допустимо. «Драфт» делает это выбором кандидата: по умолчанию работодатель видит код и опыт без текущего места работы (слепой первичный отбор и минимизация персональных данных по 152-ФЗ), а кандидат одним переключателем открывает имя и текущую компанию.

4. СОБСТВЕННАЯ ПРОЦЕДУРА ВАЛИДАЦИИ И ЕЁ РЕЗУЛЬТАТЫ

Отчёт сформирован 2026-10-08 10:07 командой `python -m app.validation.run_all` (≈ 10 секунд), исходные данные — `backend/validation_results/report.json`, интерактивная версия — страница **/validation** в приложении.

4.1. Идея: синтетическая популяция со скрытой «истиной»

Эталонных датасетов и разметки в задаче нет. Поэтому мы строим популяцию, где **истина известна, но скрыта от системы**:

- у каждого синтетического кандидата есть истинная способность θ, истинный грейд и **реальный** набор навыков;
- наблюдаемые данные зашумлены, как в жизни: **25% завышают** грейд на ступень, 8% занижают, **20% «набивают» резюме** навыками, которыми не владеют, 10% заявленных навыков теряются;
- ответы на задания моделируются 3PL-моделью с **«истинными» параметрами, отличными от калибровочных** (шум ±0.3 по b, ±15% по a) — система работает с неточными параметрами, как в реальности до калибровки;
- у 36% кандидатов есть история ФСП из синтетического реестра; сила спортсмена коррелирует с его θ, а стек его результатов — с реальными навыками;
- тест кандидата в базе пройден **тем же CAT-движком**, что и у живых пользователей.

Система видит только наблюдаемое; качество измеряется сравнением с истиной. Это отвечает на главный вопрос Т3: помогает ли верифицированный профиль находить подходящих людей лучше, чем самоописанное резюме.

4.2. Механика тестирования (Монте-Карло, 1200 виртуальных кандидатов, каждый проходит тест дважды)

Конфигурация: **классификационная остановка** — тест завершается, как только по каждому порогу грейда апостериорная вероятность «выше» или «ниже» достигла 85%; от 8 до 22 заданий (медиана — 14, 10-й перцентиль — 8, 90-й — 22; дольше 14 заданий тест длится у 47% — это пограничные случаи).

Точность категоризации (соответствие эталону)

Метрика	Значение
Грейд по тесту совпадает с истинным	**81%** (заявленный самим кандидатом — 74%)
Ошибка не больше одной ступени	100%
Точное совпадение, если истинная θ дальше 0.25 от порога грейда	90%
κ Коэна / взвешенная κ	0.74 / 0.90
Корреляция $\hat{\theta}$ с истинной θ, RMSE, смещение	0.940, 0.39, +0.003
Решение «грейд подтверждён»: чувствительность / специфичность	92% / 77%
Среднее число заданий / средняя SE	14.8 / 0.41
Завысили грейд в анкете (19% популяции): тест **не подтвердил** завышенный грейд	**85%** (точно вернул к истинному — 77%)
Заявили грейд честно: тест подтвердил заявленный	82%

Матрица ошибок (строки — истинный грейд, столбцы — грейд по тесту):

	Стажёр	Junior	Middle	Senior
Стажёр	129	32	0	0
Junior	32	272	54	0
Middle	0	34	313	46
Senior	0	0	30	258

Все ошибки — на соседний грейд и сосредоточены у порогов, где ±1 стандартная ошибка измерения пересекает границу полосы. Точность в зависимости от расстояния истинной θ до ближайшего порога:

Расстояние до порога (логиты)	Доля популяции	Точность грейда
0–0.1	15%	56%
0.1–0.25	23%	72%
0.25–0.5	38%	86%
> 0.5	24%	98%

У кандидата, чья способность лежит ровно на границе Junior/Middle, «правильного» грейда по сути нет — любой тест разумной длины здесь ошибается почти в половине случаев. Политика «грейд не понижается принудительно» + пересдача через 14 дней снимает остаточный риск для кандидата.

Правило остановки: по точности оценки или по надёжности решения

Тест решает задачу классификации, поэтому мы сравнили прежнее правило (остановка при SE ≤ 0.35, 10–16 заданий, задания по информации в $\hat{\theta}$) с классификационным (остановка, когда решение по каждому порогу надёжно; у границы грейда задания подбираются по информации в точке порога; пограничным — до 22 заданий). Одна и та же популяция из 1000 кандидатов, два независимых прохождения:

Правило	Точный грейд	Вдали от порогов	Решение «подтверждён»	Повтор: тот же грейд	Заданий в среднем (макс.)
По точности (SE ≤ 0.35), прежнее	81%	88%	89%	73%	15.4 (16)
По надёжности решения (Драфт)	83%	90%	91%	75%	14.6 (22)

Классификационная остановка точнее и при этом **короче в среднем**: очевидные случаи завершаются за 8–12 заданий, а сэкономленное время уходит на спорные.

Категоризация по стеку: разбор резюме на размеченном вручную наборе

Специализацию и навыки из резюме система предлагает кандидату при заполнении профиля (грейд затем подтверждает тест). Чтобы проверить разбор не на собственных синтетических текстах, мы написали два набора резюме в разных стилях (hh.ru, LinkedIn, свободный текст; русский и английский; сленг «реакт», «k8s», «докер»; разные форматы контактов и дат) и разметили ФИО, контакты, город, специализацию, стаж и навыки онтологии: **основной** (24 резюме, 234 навыков) — по нему дорабатывался парсер, и **отложенный** (16 резюме, 131 навыков) — написан после доработки; первый замер на нём сделан до любых правок.

Поле	Основной набор	Отложенный: первый замер	Отложенный: после правок
ФИО	100%	94%	100%
E-mail	100%	100%	100%
Телефон	100%	100%	100%
Telegram	100%	100%	100%
GitHub	100%	100%	100%

Поле	Основной набор	Отложенный: первый замер	Отложенный: после правок
Город	100%	100%	100%
Специализация	96%	94%	94%
Стаж (±0.5 года)	100%	81%	100%
Навыки: precision / recall / F1	0.98 / 1.00 / **0.99**	0.97 / 0.91 / **0.94**	0.98 / 0.97 / **0.97**

Оставшиеся ошибки — содержательные неоднозначности: fullstack PHP + Vue относится к frontend, C++-разработчик сетевого шлюза — к информационной безопасности, MVVM засчитывается как «чистая архитектура». Кандидат подтверждает распознанное перед сохранением, а категорию определяет тест — ошибка разбора резюме не меняет грейд.

Согласованность (test-retest на разных формах)

Метрика	Значение
Корреляция $\hat{\theta}$ двух прохождений	**0.882**
Совпадение грейда / к / взвешенная к	73% / 0.63 / 0.85
Совпадение решения «подтверждён / нет»	86%
Грейды различаются не больше чем на ступень	100%
Доля общих семейств в двух формах одного кандидата	36% (варианты при этом разные)

Дискриминативность

Метрика	Значение
AUC разделения соседних грейдов по $\hat{\theta}$	intern→junior: 0.917, junior→middle: 0.943, middle→senior: 0.941
Средняя $\hat{\theta}$ по истинным грейдам	Стажёр -1.42, Junior -0.46, Middle +0.50, Senior +1.58
Медиана точечно-бисериальной корреляции задания с $\hat{\theta}$ (по 197 семействам)	0.530; доля семейств с r > 0.2 — 98%

Устойчивость к утечке заданий

Эксперимент	Результат
Уникальность вариантов (150 генераций на семейство)	медиана доли уникальных — 91%; вероятность совпадения двух вариантов — 0.87%
Атака: Junior (θ = −0.5) сдаёт Middle **без утечки**	проходит 7%, $\hat{\theta}$ = -0.47
…**статичный банк**, утекло 60% ответов	проходит **97%**, $\hat{\theta}$ завышена на **+1.18**
…параметрический банк **без проверки свежести**: известны варианты 40 предыдущих кандидатов	проходит 18%, $\hat{\theta}$ завышена на +0.14; из утечки отвечено 9% заданий
…**«Драфт»**: то же, но сервер не выдаёт уже показанный вариант	проходит **5%** (как без утечки), $\hat{\theta}$ сдвинута на **-0.03**; из утечки отвечено **0%** заданий
Максимальная экспозиция семейства (узкая популяция, Backend)	randomesque: 49%; без контроля (максимум информации): 100%
Пересечение форм двух кандидатов	18% против 33% без контроля экспозиции
Смешанная популяция всех специализаций	макс. экспозиция 29%, задействовано 225 семейств

Контрольная форма при сигналах нечестности

Если попытка прошла порог, но индекс согласованности ответов lz < -2.5 или сработали два поведенческих сигнала (уходы со вкладки, вставки, сверхбыстрые верные ответы на сложные задания), грейд не присваивается до короткой контрольной формы: 6–10 свежих заданий у порога, таймер строже на 30%.

Измерение (симуляция)	Результат
Честные кандидаты (800), отправлены на контрольную форму по lz	**0.6%**
«Помощь извне» на половине заданий (θ = −0.8, 300 попыток): проходят Middle	86%
…из них распознаны по lz	5%

Вывод: статистический детектор почти не мешает честным, но живую подсказку в коротком адаптивном тесте ловит плохо — это задача прокторинга, а не модели. От **утечки** (заученных чужих вариантов) защищает гарантия свежих вариантов выше; поведенческие сигналы детектора оцениваются только на реальных попытках.

Адаптивный тест против фиксированной формы той же средней длины (14 заданий)

	Точность грейда	Средняя SE
CAT «Драфта»	**81%**	0.42
Фиксированная форма	71%	0.52

Онлайн-калибровка и детекция утечки «способа решения»

Метрика	Значение
Откалибровано семейств (≥ 30 ответов)	181
RMSE трудности b: априорная → после калибровки	0.299 → **0.192**
Имитация: 8 семейств стали легче на 1.2 логита	найдено 7 из 8, ложных срабатываний 6 из 181

4.3. Механика подбора: тестовый набор пар «потребность — кандидат»

Набор: 217 потребностей (5 независимых наборов по 8 специализациям × 4 грейдам; стек обязательных и желательных навыков — из реальных стеков треков, вилка — около рыночной медианы, формат — случайный) × 620 опубликованных кандидатов, которые рассматривают предложения.

Эталонная релевантность (0–3) по скрытой истине:

- 0 — другая специализация, истинный грейд дальше ступени, нет ни одного обязательного навыка;
- 1 — соседний грейд ниже, слабое покрытие обязательных навыков (< 1/3);
- 2 — истинный грейд совпадает и навыки покрыты; или кандидат **сильнее запроса на ступень и укладывается в бюджет**;
- 3 — истинный грейд совпадает, реально владеет ≥ 2/3 обязательных навыков и находится в верхней половине грейда;
- −1 — ожидания выше вилки более чем на 10%; −1 — несовместимый формат работы.

Метрики: Precision@10 (доля релевантных ≥ 2 в топ-10), nDCG@10 (gain = 2^rel − 1), MRR, Recall@30, «шум» — доля полностью нерелевантных в показанных топ-10.

Ранжирование	P@5	P@10	nDCG@10	MRR	Recall@30	Шум@10
Поиск по ключевым словам резюме (классическая площадка)	0.586	0.520	0.538	0.780	0.689	0.159
Самоописанная категория (наша формула без верифицированного слоя)	0.732	0.619	0.681	0.911	0.864	0.072
Драфт — полная модель	0.848	0.718	0.793	0.979	0.908	0.057
Драфт с экспертными весами (до подстройки)	0.828	0.707	0.772	0.969	0.901	0.077

Ранжирование	P@5	P@10	nDCG@10	MRR	Recall@30	Шум@10
Драфт без учёта ФСП	0.844	0.714	0.790	0.977	0.907	0.057
Драфт без смежных категорий	0.844	0.715	0.787	0.979	0.775	0.045

- Разброс nDCG@10 между наборами потребностей: «Драфт» ± 0.014, классика ± 0.013 — выводы устойчивы.
- «Драфт» лучше классического поиска по резюме в **99%** запросов.
- Разница «полная модель − самоописанная категория» — вклад **верифицированного профиля** (грейд из теста, подтверждённые навыки, положение внутри грейда).
- Учёт ФСП не снижает релевантность и поднимает долю участников соревнований ФСП в топ-10 до **44%** при доле в базе 35% — требование ТЗ «выше те, у кого есть опыт ФСП» выполняется без потери качества.
- Смежные категории повышают Recall@30 — работодатель не теряет подходящих людей, когда целевая категория мала.
- **Веса формулы подобраны на отдельных наборах потребностей** (зёрна 201, 202, 203, `python -m app.validation.tune_matching`), а метрики выше посчитаны на других, независимых наборах. Подбор шёл внутри продуктовых рамок (навыки ≥ 20%, ФСП ≥ 10%, текст резюме ≤ 12%): без рамок поиск «подгонял» формулу под синтетику — увеличивал вес текста резюме и обнулял подтверждённые навыки и ФСП, что противоречит ТЗ. Итог против экспертных весов: nDCG@10 0.772 → **0.793**, P@5 0.828 → **0.848**.

Проверка на внешнем наборе пар — инструмент для жюри

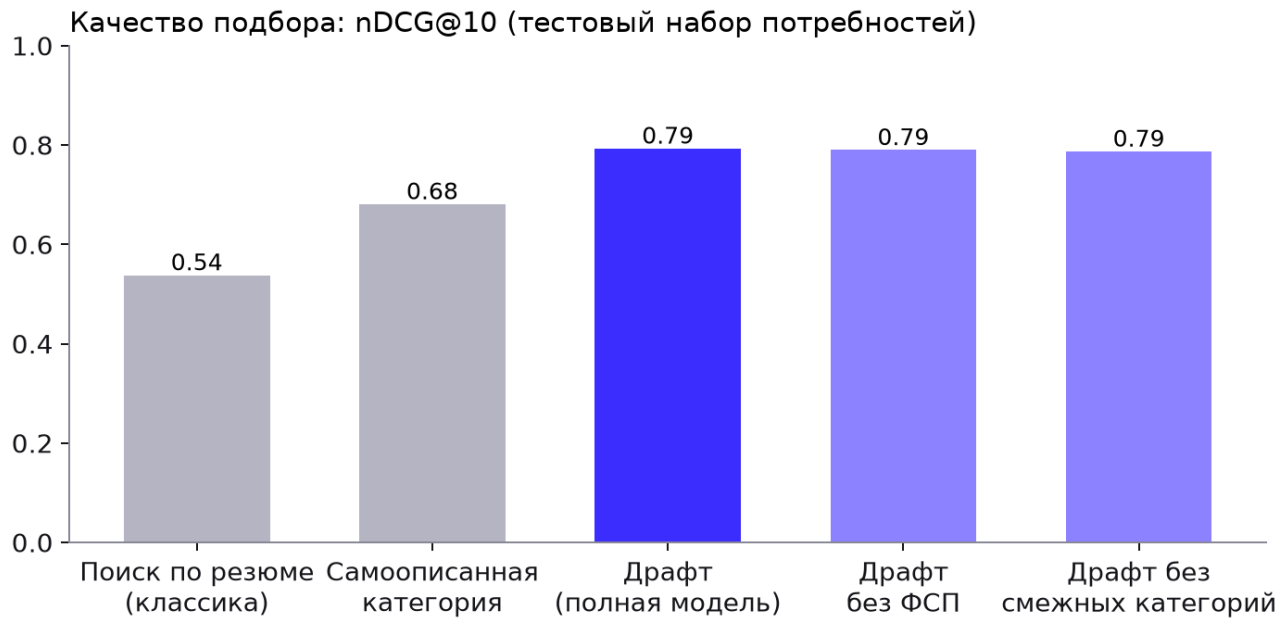
Механику можно проверить на **любом своём размеченном наборе** «вакансия — кандидат»: страница **/validation** → блок «Проверьте подбор на своём наборе пар» (загрузка JSON), API `POST /api/v1/validation/external-eval` или консоль `python -m app.validation.external_eval data.json`. Вакансии передаются текстом (разбираются тем же NLP, что и в кабинете работодателя) или полями; кандидаты — текстом резюме или полями, при наличии — с результатом теста платформы и достижениями ФСП; разметка — 0–3 или 0/1. Кандидат без теста ранжируется по заявленной категории, без ФСП — без бонуса. Два протокола: **по всей базе** (неразмеченные пары — нерелевантны, как в продукте) и **только размеченные пары**. Формат и пример — `docs/examples/eval_pairs_example.json`.

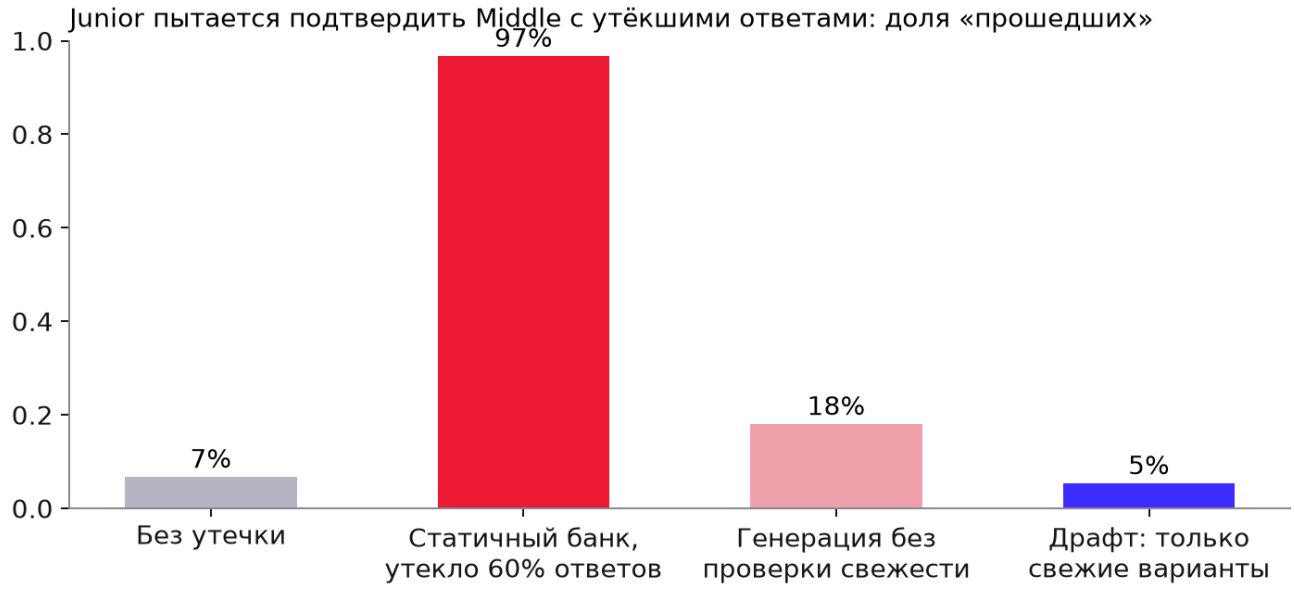
Прогон на примере (вакансии — тексты, кандидаты — тексты резюме, у 445 из 620 есть результат теста, у 218 — достижения ФСП; вакансий — 47, размеченных пар — 5728, доля релевантных пар 10%):

Протокол	Ранжирование	P@10	nDCG@10	MRR	Recall@30
по всей базе	**Драфт**	0.619	0.785	0.949	0.904
по всей базе	Поиск по резюме (классика)	0.489	0.581	0.750	0.691
по всей базе	Потолок при этой разметке	0.838	1.000	1.000	1.000
только пары	**Драфт**	0.636	0.810	0.970	0.937
только пары	Поиск по резюме (классика)	0.543	0.648	0.824	0.701
только пары	Потолок при этой разметке	0.838	1.000	1.000	1.000

nDCG@10 выше классического поиска в **42 из 47** вакансий. На пяти независимых выборках вакансий и кандидатов того же размера: nDCG@10 0.77 ± 0.02 против 0.56 ± 0.02, P@10 0.66 против 0.52. «Потолок» — максимум P@10, достижимый при числе релевантных кандидатов в разметке (у части вакансий их меньше 10).

Графики: `matching_ndcg_at_10.png`, `matching_p_at_10.png`, `testing_confusion.png`, `testing_leak_attack.png`, `testing_exposure.png`.





4.4. Как воспроизвести и как продолжить валидацию с ФСП

```
cd backend && .venv/bin/python -m app.validation.run_all          # полный прогон
cd backend && .venv/bin/python -m app.validation.run_all --fast   # быстрый прогон
```

Ограничения: параметры заданий — экспертные априорные, уточняемые онлайн-калибровкой; выводы о подборе получены на синтетических данных. План валидации на реальных данных после хакатона:

1. **Калибровка банка** на 300–500 прохождениях участников ФСП (MML-оценка a и b, анализ DIF между языковыми треками).
2. **Конвергентная валидность**: корреляция θ с рейтингом ФСП и местами на соревнованиях.
3. **Экспертная разметка подбора**: 20 потребностей × 30 кандидатов, разметка 2–3 рекрутерами (согласованность — κ), расчёт тех же P@10/nDCG@10 встроенным инструментом `app.validation.external_eval` (формат — `docs/examples`).
4. **A/B-тест в продукте**: доля принятых приглашений и время до найма — с подборкой «Драфта» и без неё.

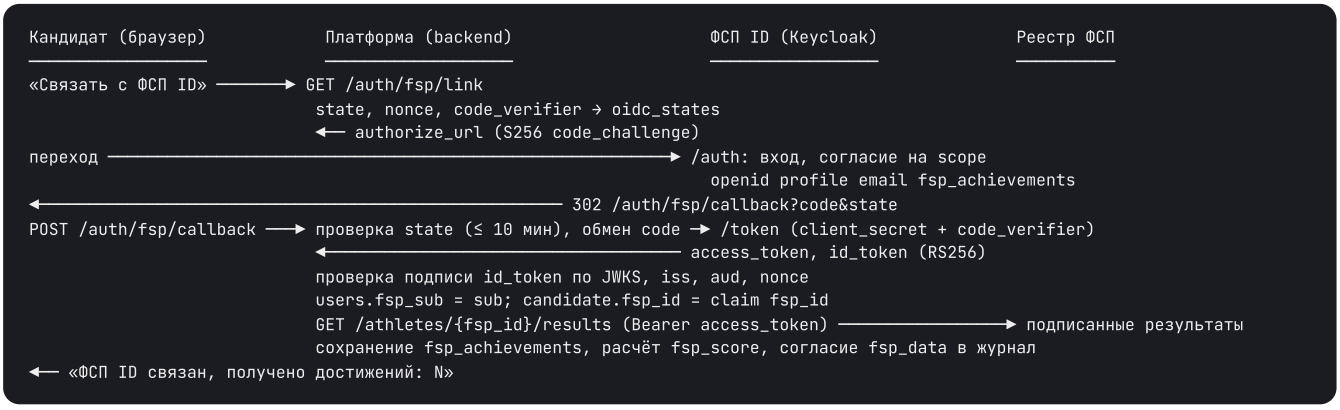
5. СХЕМА ИНТЕГРАЦИИ С ФСП ID И РЕЕСТРОМ ДОСТИЖЕНИЙ ФСП

API и структура данных ФСП в рамках хакатона намеренно не предоставлялись. Мы спроектировали интеграцию так, чтобы переход на настоящие сервисы ФСП требовал **только смены адресов и секретов**, а не изменения кода платформы.

5.1. Состав

Компонент	В прототипе	В продакшене
ФСП ID	`fsp-mock`: OIDC-провайдер с URL и форматом токенов Keycloak (`/realms/fsp/protocol/openid-connect/…`), RS256, JWKS, PKCE	Keycloak ФСП (реалм `fsp`), клиент `draft-platform` (confidential)
Реестр достижений	`fsp-mock`: `/api/v1/athletes/{fsp_id}`, `/api/v1/athletes/{fsp_id}/results`, `/api/v1/events`	сервис реестра ФСП с тем же контрактом (предложение команды)
Платформа	`backend/app/fsp/client.py` — discovery, авторизация, обмен кода, проверка `id_token` по JWKS, загрузка результатов	без изменений

5.2. Поток связывания профиля (Authorization Code + PKCE)



Тот же поток с `intent=login` позволяет **войти или зарегистрироваться через ФСП ID**: если `sub` уже связан — выдаются токены платформы; если нет — создаётся аккаунт кандидата с подтверждённым e-mail из ФСП ID (при совпадении e-mail с существующим аккаунтом связь не делается автоматически — защита от захвата аккаунта; пользователь входит паролем и связывает в настройках).

Фоновая синхронизация: `POST /candidate/fsp/sync` получает сервисный токен (`client_credentials`, роль `registry-reader`) и обновляет результаты. В продакшене — по расписанию и по событию реестра (вебхук «новые результаты соревнования»).

5.3. Предлагаемый контракт реестра достижений

```
GET /api/v1/athletes/FSP-2186491/results
{
  "athlete": {"fsp_id": "FSP-2186491", "full_name": "Егор Ильин", "region": "Калининградская область",
              "sport_rank": "КМС", "rating": 1834, "club": "Клуб спортивного программирования «Kernel»"},
  "results": [{
    "result_id": "R-EV-2026-021-6491",
    "event": {"event_id": "EV-2026-021", "name": "Чемпионат СЗФО по продуктовому программированию 2026",
              "discipline": "product", "level": "interregional", "start_date": "2026-03-14", "city": "Санкт-Петербург",
              "format": "team", "participants": 104},
    "place": 2, "units_total": 26, "percentile": 0.962,
    "team_name": "KernelLambda", "team_role": "Backend-разработчик",
    "score": 91.4, "stack": ["Python", "FastAPI", "PostgreSQL", "Docker"],
    "diploma_url": "https://…/R-EV-2026-021-6491.pdf",
    "signature": "<подпись записи реестром>"
  }],
  "schema_version": "1.0"
}
```

Почему именно эти поля (что полезно работодателю):

Поле	Зачем
`discipline` (5 официальных дисциплин: алгоритмическое, продуктовое, ИБ, робототехника, БАС)	показывается работодателю; рейтинга дисциплин нет, учитывается только **актуальность дисциплины для специализации** (профильная 1.0, смежная 0.8, непрофильная 0.6)
`level` (региональный, межрегиональный, всероссийский, международный)	показывается работодателю; в оценку **не входит** — «чемпионат России» не весит больше хакатона
`place`, `units_total`, `percentile`	результативность (победитель 1.0, призёр 0.85/0.75, финалист топ-10% 0.55, топ-25% 0.4, участник 0.2)
`team_role`, `stack`	**подтверждение конкретных навыков**: стек решения — доказательство силы 0.9 в подборе
`date`	затухание давности (полупериод 2 года)
`signature`, `diploma_url`	проверяемость записи и диплома
`sport_rank`, `rating`	спортивный разряд и рейтинг ФСП в профиле

Оценка (`app/fsp/scoring.py`):

```
вклад результата = фактор результата × давность × актуальность дисциплины
Σ = Σ вклады призовых и финальных результатов + min(Σ вклады участий без результата, 0.6)
fsp_score = 1 - exp(-Σ / 2) ∈ [0, 1]
```

Решения, согласованные с постановщиками на сессии вопросов и ответов и в письменных ответах организаторов:

- **Все соревнования и дисциплины ФСП равноценны** — внутреннего рейтинга турниров и дисциплин нет, уровень соревнования отображается, но в оценку не входит.
- **Актуальность дисциплины для специализации** (идея организаторов из письменных ответов: специалист по ИБ с соревнованием по ИБ релевантнее, чем с соревнованием по робототехнике). Это не рейтинг дисциплин, а соответствие дисциплины профессии: результат в профильной дисциплине учитывается полностью, в смежной — с весом 0.8, в непрофильной — 0.6 (любой результат ФСП засчитывается). Если стек решения совпал с навыками запроса работодателя, результат считается профильным независимо от дисциплины. В профиле актуальность считается к специализации кандидата, в подборке — к специализации и навыкам запроса; в объяснении выдачи написано, профильная ли дисциплина.

Дисциплина	Профильная для	Смежная для	Непрофильная для
Продуктовое программирование	Backend, Frontend, Mobile	ML, аналитика, DevOps, QA, ИБ	—
Алгоритмическое программирование	Backend, ML	Frontend, Mobile, аналитика, ИБ	DevOps, QA

Дисциплина	Профильная для	Смежная для	Непрофильная для
Программирование систем ИБ	ИБ	Backend, DevOps, QA	Frontend, Mobile, ML, аналитика
Робототехника	—	Backend, Mobile, ML	Frontend, аналитика, DevOps, QA, ИБ
Беспилотные авиационные системы	—	Backend, Mobile, ML	Frontend, аналитика, DevOps, QA, ИБ
* **Учитываются результативность и число соревнований**: каждое призовое или финальное место добавляет вклад.			
* **Профиль нельзя «набить» онлайн-явками**: участия без результата суммируются с потолком 0.6 — любое их число весит			
меньше одного призового места (победитель — 1.0, призёр — 0.75–0.85).			
* **Срок давности**: вклад результата убывает вдвое за 2 года — свежий результат котируется сильнее.			
* **ФСП не влияет на грейд**: грейд присваивает только тест; `fsp_score` двигает кандидата внутри категории.			

5.4. Кандидат без истории ФСП

Обязательное требование ТЗ — корректная обработка отсутствия истории:

- категория определяется **только тестом** — отсутствие ФСП на неё не влияет;
- в ранжировании компонента F = 0: участники ФСП получают бонус, но кандидат без истории **не штрафуется и не исключается** (проверено автотестом `test_candidate_without_fsp_handled` и в валидации);
- в карточке работодателя — пояснение «Кандидат пока не участвовал в соревнованиях ФСП или не связал ФСП ID — это не влияет на категорию»;
- кандидату — мягкое предложение связать ФСП ID; если ФСП ID связан, но результатов нет, — отдельное сообщение.

5.5. Переход на настоящий ФСП ID (Keycloak)

1. В реалме ФСП создать confidential-клиент `draft-platform`: Standard Flow + PKCE (S256), Valid Redirect URI `https://<домен платформы>/auth/fsp/callback`, Service Accounts (для синхронизации).
2. Добавить client scope `fsp_achievements` и protocol mapper атрибута пользователя `fsp_id` в access и id token.
3. Реестру достижений проверять токены Keycloak (audience `fsp-registry`) и отдавать данные только владельцу (`fsp_id` из токена) или сервисному аккаунту с ролью `registry-reader`.
4. На платформе задать переменные:

```
FSP_OIDC_PUBLIC_BASE=https://id.fsp-russia.ru        # адрес Keycloak для браузера
FSP_OIDC_INTERNAL_BASE=https://id.fsp-russia.ru      # адрес для backend (может совпадать)
FSP_REALM=fsp
FSP_CLIENT_ID=draft-platform
FSP_CLIENT_SECRET=<секрет клиента>
FSP_REGISTRY_API=https://registry.fsp-russia.ru/api/v1
```

Платформа использует стандартный discovery (`/.well-known/openid-configuration`), поэтому других изменений не требуется.

Совместимость собственных токенов платформы

Access-токены платформы подписываются RS256 и повторяют структуру Keycloak: `iss`, `sub`, `aud`, `azp`, `typ`, `realm_access.roles`, `email_verified`, `preferred_username`; публичный ключ доступен в формате JWKS (`/api/v1/auth/jwks`). Поэтому при желании ФСП платформа может полностью перейти на Keycloak как единый провайдер: бизнес-код читает те же claims, меняется только источник токена (issuer + JWKS URL).

6. ОПИСАНИЕ API

Все методы описаны спецификацией **OpenAPI 3** — интерактивно: `/api/docs` (Swagger UI) и `/api/redoc`, файл: `docs/openapi.json`. Всего 119 операций.

Общие правила

- Базовый путь: `/api/v1`. Формат — JSON, кодировка UTF-8. Даты — ISO 8601 (UTC).
- Аутентификация: `Authorization: Bearer <access_token>` (JWT RS256, 30 минут). Обновление — `POST /auth/refresh` с ротацией refresh-токена (14 дней).
- Роли: `candidate`, `employer`, `admin`. Доступ к кабинетам — только с подтверждённым e-mail. ATS-методы — по заголовку `X-API-Key`.
- Ошибки — единый формат с машиночитаемым кодом:

```
{"error": {"code": "not_eligible", "message": "Смена грейда возможна не чаще одного раза в 90 дней", "details": {"available_at": "2027-01-03T10:00:00+00:00"}}}
```

Код	Значение
400	некорректный запрос (`invalid_code`, `invalid_salary`, …)
401	нет или недействителен токен (`unauthorized`, `token_expired`, `invalid_credentials`)
403	нет прав (`forbidden`) или не подтверждён e-mail (`email_not_verified`)
404	объект не найден или принадлежит другому пользователю (`not_found`)
409	конфликт состояния (`already_invited`, `not_eligible`, `attempt_closed`, `stale_item`, `email_taken`)
422	ошибка валидации (`validation_error` со списком полей) или признаки мошенничества (`risky_content`)
423	учётная запись временно заблокирована после 5 неудачных входов
429	превышен лимит запросов или суточный лимит приглашений (`rate_limited`, `invite_limit`)
503	недоступен внешний сервис ФСП (`fsp_unavailable`)

Ключевые сценарии

```
# регистрация кандидата → письмо с кодом → подтверждение (возвращает токены)
curl -X POST /api/v1/auth/register -d
'{"email":"a@b.ru","password":"Secret123","role":"candidate","consent_pd":true,"consent_publication":true}'
curl -X POST /api/v1/auth/verify-email -d '{"email":"a@b.ru","code":"354777"}'
# опрос → тест → ответы
curl -X POST /api/v1/candidate/survey -d '{"direction":"development","specialization":"backend","primary_lang":"python","stack":
["Python","PostgreSQL"],"experience_years":4,"declared_grade":"middle"}'
curl -X POST /api/v1/candidate/assessment/attempts -d '{"target_grade":"middle"}'          # → attempt_id + первое задание
curl -X POST /api/v1/candidate/assessment/attempts/42/answer -d '{"item_id":901,"response":"3","time_ms":41000,"events":
{"blur":0}}'
# работодатель: потребность → подборка → приглашение
curl -X POST /api/v1/employer/needs/parse -d '{"text":"Ищем Middle Python-разработчика… 220 000 – 300 000 ₽"}'
curl -X POST /api/v1/employer/needs -d '{"title":"Python в биллинг","specialization":"backend","grades":["middle"],"must_skills":
["Python","PostgreSQL"],"salary_from":220000,"salary_to":300000}'
curl '/api/v1/employer/needs/1/shortlist?skills=PostgreSQL&verified_only=true&within_budget=true'
curl -X POST /api/v1/employer/invitations -d '{"candidate_ids":[57],"title":"Python-
разработчик","description":"…","salary_from":250000,"salary_to":300000,"contact_method":"hr@company.ru","need_id":1}'
# кандидат: принять приглашение (контакты раскрываются работодателю)
curl -X POST /api/v1/candidate/invitations/15/accept -d '{"message":"Готов к созвону"}'
```

Аутентификация

Метод	Путь	Назначение	Успех	Ошибки
`POST`	`/api/v1/auth/register`	Регистрация по e-mail	201	409, 422

Метод	Путь	Назначение	Успех	Ошибки
`POST`	`/api/v1/auth/verify-email`	Подтверждение e-mail кодом или токеном из письма	200	400, 422
`POST`	`/api/v1/auth/resend-verification`	Повторно отправить письмо с кодом	202	422
`POST`	`/api/v1/auth/login`	Вход по e-mail и паролю	200	401, 422, 423
`POST`	`/api/v1/auth/refresh`	Обновить access-токен (ротация refresh-токена)	200	401, 422
`POST`	`/api/v1/auth/logout`	Выход (отзыв refresh-токена)	204	422
`GET`	`/api/v1/auth/me`	Текущий пользователь	200	401, 403, 422
`GET`	`/api/v1/auth/jwks`	Публичные ключи подписи токенов (JWKS)	200	
`GET`	`/api/v1/auth/.well-known/openid-configuration`	OIDC discovery платформы (Keycloak-совместимые claims)	200	
`GET`	`/api/v1/auth/fsp/login`	Начать вход/регистрацию кандидата через ФСП ID	200	503
`GET`	`/api/v1/auth/fsp/link`	Начать связывание профиля кандидата с ФСП ID	200	401, 403, 422
`POST`	`/api/v1/auth/fsp/callback`	Завершение OIDC-авторизации ФСП ID	200	400, 409, 422

Кандидат

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/candidate/me`	Профиль кандидата с категорией, статусом и шагами онбординга	200	401, 403, 422
`PUT`	`/api/v1/candidate/profile`	Сохранить профиль/резюме (ручное заполнение или подтверждение распознанного PDF)	200	401, 403, 422
`POST`	`/api/v1/candidate/resume/parse`	Загрузка PDF-резюме с авторазпознаванием полей	200	400, 401, 403, 413, 422
`POST`	`/api/v1/candidate/resume/parse-text`	Распознавание резюме из текста (вставка из буфера)	200	401, 403, 422
`GET`	`/api/v1/candidate/profile/pdf`	Стандартизированный PDF-профиль с QR-кодом проверки	200	401, 403, 422
`POST`	`/api/v1/candidate/survey`	Опрос по отрасли и специализации (обязательный шаг перед тестом)	200	401, 403, 409, 422
`GET`	`/api/v1/candidate/categories`	Мои категории: основная и дополнительные	200	401, 403, 422
`POST`	`/api/v1/candidate/categories`	Добавить дополнительную категорию (грейд заявленный — подтверждается тестом)	201	401, 403, 409, 422
`POST`	`/api/v1/candidate/categories/{spec}/main`	Сделать дополнительную категорию основной — чтобы пройти по ней тест	200	401, 403, 404, 409, 422

Метод	Путь	Назначение	Успех	Ошибки
`DELETE`	`/api/v1/candidate/categories/{spec}`	Удалить дополнительную категорию	200	401, 403, 404, 422
`GET`	`/api/v1/candidate/history`	История опросов, попыток тестирования и изменений грейда	200	401, 403, 422
`GET`	`/api/v1/candidate/insights`	Зарплатный радар и путь роста по данным категории (обезличенно)	200	401, 403, 422
`GET`	`/api/v1/candidate/invitations`	Входящие приглашения от работодателей (только свои)	200	401, 403, 422
`GET`	`/api/v1/candidate/invitations/{inv_id}`	Приглашение (при открытии отмечается как просмотренное)	200	401, 403, 422
`POST`	`/api/v1/candidate/invitations/{inv_id}/accept`	Принять приглашение — контакты раскрываются работодателю	200	401, 403, 422
`POST`	`/api/v1/candidate/invitations/{inv_id}/decline`	Отклонить приглашение с причиной (обратная связь работодателю)	200	401, 403, 422
`POST`	`/api/v1/candidate/invitations/{inv_id}/contacts`	Закрыть или снова открыть работодателю доступ к своим контактам	200	401, 403, 409, 422
`GET`	`/api/v1/candidate/invitations/{inv_id}/messages`	Переписка с работодателем по принятому приглашению	200	401, 403, 422
`POST`	`/api/v1/candidate/invitations/{inv_id}/messages`	Написать работодателю (после принятия приглашения)	201	401, 403, 409, 422, 429
`POST`	`/api/v1/candidate/invitations/{inv_id}/report`	Пожаловаться на приглашение (антифрод)	201	401, 403, 422
`GET`	`/api/v1/candidate/applications`	Мои отклики и их статусы	200	401, 403, 422
`POST`	`/api/v1/candidate/applications`	Откликнуться на вакансию (контакты раскрываются работодателю)	201	401, 403, 404, 409, 422
`DELETE`	`/api/v1/candidate/applications/{app_id}`	Отозвать отклик	200	401, 403, 422
`GET`	`/api/v1/candidate/tasks`	Текущая регулярная задача и история	200	401, 403, 422
`POST`	`/api/v1/candidate/tasks/request`	Получить новую задачу сейчас (если нет активной)	200	401, 403, 422
`POST`	`/api/v1/candidate/tasks/{assignment_id}/submit`	Отправить решение или подход к решению	200	401, 403, 422
`POST`	`/api/v1/candidate/tasks/{assignment_id}/skip`	Пропустить задачу	200	401, 403, 422
`PUT`	`/api/v1/candidate/settings/privacy`	Настройки приватности и видимости профиля	200	401, 403, 409, 422
`GET`	`/api/v1/candidate/consents`	Текущие согласия и журнал изменений	200	401, 403, 422

Метод	Путь	Назначение	Успех	Ошибки
`POST`	`/api/v1/candidate/consents`	Дать или отозвать согласие	200	401, 403, 422
`POST`	`/api/v1/candidate/fsp/sync`	Обновить достижения из реестра ФСП (сервисный токен реестра)	200	401, 403, 422, 503
`DELETE`	`/api/v1/candidate/fsp`	Отвязать ФСП ID и удалить полученные достижения	200	401, 403, 422
`GET`	`/api/v1/candidate/export`	Выгрузка всех персональных данных (право субъекта ПДн, 152-ФЗ ст. 14)	200	401, 403, 422
`DELETE`	`/api/v1/candidate/account`	Удалить аккаунт и все персональные данные	204	401, 403, 422

Тестирование

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/candidate/assessment/eligibility`	На какие грейды можно пройти тест сейчас (с учётом ограничений по частоте)	200	401, 403, 422
`POST`	`/api/v1/candidate/assessment/attempts`	Начать тест на выбранный грейд (возвращает первое задание)	201	401, 403, 409, 422
`GET`	`/api/v1/candidate/assessment/attempts/{attempt_id}`	Состояние попытки: текущее задание или итог	200	401, 403, 422
`POST`	`/api/v1/candidate/assessment/attempts/{attempt_id}/answer`	Ответ на текущее задание; возвращает следующее задание или итог	200	401, 403, 409, 422
`POST`	`/api/v1/candidate/assessment/attempts/{attempt_id}/expert/answer`	Режим эксперта (демо): ответить на текущее задание верно или неверно	200	401, 403, 422

Метод	Путь	Назначение	Успех	Ошибки
`POST`	`/api/v1/candidate/assessment/attempts/{attempt_id}/expert/simulate`	Режим эксперта (демо): пройти оставшиеся задания как кандидат уровня θ (модель IRT 3PL)	200	401, 403, 422
`POST`	`/api/v1/candidate/assessment/attempts/{attempt_id}/finish`	Досрочно завершить попытку (оставшиеся задания не засчитываются)	200	401, 403, 422
`POST`	`/api/v1/candidate/assessment/attempts/{attempt_id}/upgrade`	Начать отдельный тест на грейд выше по предложению из результата (без ожидания 90 дней, вне лимита попыток)	201	401, 403, 409, 422

Работодатель

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/employer/company`	Профиль компании и индекс доверия	200	401, 403, 422
`PUT`	`/api/v1/employer/company`	Обновить профиль компании	200	401, 403, 422
`GET`	`/api/v1/employer/dashboard`	Сводка: потребности, приглашения, отклики, задачи	200	401, 403, 422
`POST`	`/api/v1/employer/needs/parse`	Разбор текстового описания потребности (NLP): специализация, грейд, стек, вилка, формат	200	401, 403, 422
`GET`	`/api/v1/employer/needs`	Мои потребности	200	401, 403, 422
`POST`	`/api/v1/employer/needs`	Описать потребность — система сформирует подборку кандидатов	201	401, 403, 422
`GET`	`/api/v1/employer/needs/{need_id}`	Потребность и рекомендованные категории	200	401, 403, 422

Метод	Путь	Назначение	Успех	Ошибки
`PUT`	`/api/v1/employer/needs/{need_id}`	Изменить потребность (подборка пересчитывается, пометки сохраняются)	200	401, 403, 422
`DELETE`	`/api/v1/employer/needs/{need_id}`	Архивировать потребность	200	401, 403, 422
`POST`	`/api/v1/employer/needs/{need_id}/run`	Обновить подборку (новые кандидаты, свежие результаты)	200	401, 403, 422
`GET`	`/api/v1/employer/needs/{need_id}/shortlist`	Подборка с уточняющими фильтрами (без потери исходной подборки)	200	401, 403, 422
`GET`	`/api/v1/employer/needs/{need_id}/compare`	Сравнить 2–3 кандидатов по потребности: компоненты оценки и «почему A выше B»	200	400, 401, 403, 422
`PUT`	`/api/v1/employer/needs/{need_id}/shortlist/{candidate_id}`	Пометить кандидата в подборке (избранное/скрыть/заметка)	200	401, 403, 422
`GET`	`/api/v1/employer/needs/{need_id}/test-plan`	Как система тестирует кандидатов под эту потребность: план и примеры заданий	200	401, 403, 422
`GET`	`/api/v1/employer/categories`	Карта категорий: специализации × грейды, объём, доля ФСП, медианные ожидания	200	401, 403, 422
`GET`	`/api/v1/employer/candidates`	Поиск по банку кандидатов с фильтрами	200	401, 403, 422
`GET`	`/api/v1/employer/candidates/{candidate_id}`	Профиль кандидата (контакты — только после принятия приглашения/отклика)	200	401, 403, 422
`POST`	`/api/v1/employer/invitations`	Пригласить кандидата(ов): описание, вилка (обязательно), способ связи	201	401, 403, 409, 422, 429
`GET`	`/api/v1/employer/invitations`	Мои приглашения и их статусы	200	401, 403, 422
`GET`	`/api/v1/employer/invitations/{inv_id}`	Приглашение	200	401, 403, 422

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/employer/invitations/{inv_id}/messages`	Переписка с кандидатом по принятому приглашению	200	401, 403, 422
`POST`	`/api/v1/employer/invitations/{inv_id}/messages`	Написать кандидату (после принятия приглашения)	201	401, 403, 409, 422, 429
`POST`	`/api/v1/employer/invitations/{inv_id}/withdraw`	Отозвать неотвеченное приглашение	200	401, 403, 422
`PUT`	`/api/v1/employer/invitations/{inv_id}/stage`	Этап после принятия: знакомство → интервью → оффер → выход	200	401, 403, 422
`GET`	`/api/v1/employer/analytics`	Аналитика приглашений: конверсия, причины отказов, воронка	200	401, 403, 422
`GET`	`/api/v1/employer/vacancies`	Мои вакансии	200	401, 403, 422
`POST`	`/api/v1/employer/vacancies`	Опубликовать вакансию (зарплатная вилка обязательна)	201	401, 403, 422
`GET`	`/api/v1/employer/vacancies/{vid}`	Вакансия	200	401, 403, 422
`PUT`	`/api/v1/employer/vacancies/{vid}`	Редактировать вакансию	200	401, 403, 422
`GET`	`/api/v1/employer/vacancies/{vid}/applications`	Отклики на вакансию (контакты откликнувшихся раскрыты)	200	401, 403, 422
`PUT`	`/api/v1/employer/applications/{app_id}`	Статус отклика: просмотрен, интервью, оффер, отказ	200	401, 403, 422
`GET`	`/api/v1/employer/vacancies/{vid}/recommended`	Рекомендованные кандидаты под вакансию (пригласить напрямую)	200	401, 403, 422
`GET`	`/api/v1/employer/vacancies/{vid}/test-plan`	План тестирования кандидатов под вакансию и примеры сгенерированных заданий	200	401, 403, 422
`GET`	`/api/v1/employer/tasks`	Мои задачи для регулярных заданий	200	401, 403, 422
`POST`	`/api/v1/employer/tasks`	Создать короткую задачу (система будет предлагать её кандидатам категории)	201	401, 403, 422

Метод	Путь	Назначение	Успех	Ошибки
`PUT`	`/api/v1/employer/tasks/{tid}`	Изменить задачу	200	401, 403, 422
`GET`	`/api/v1/employer/tasks/{tid}`	Задача и решения кандидатов	200	401, 403, 422
`POST`	`/api/v1/employer/tasks/{tid}/submissions/{aid}/review`	Оценить решение (1–5) — влияет на актуальность профиля кандидата	200	401, 403, 422
`GET`	`/api/v1/employer/integrations`	API-ключи и вебхуки для интеграции с ATS	200	401, 403, 422
`POST`	`/api/v1/employer/api-keys`	Создать API-ключ (показывается один раз)	201	401, 403, 422
`DELETE`	`/api/v1/employer/api-keys/{kid}`	Отозвать API-ключ	200	401, 403, 422
`POST`	`/api/v1/employer/webhooks`	Подписать URL на события (подпись HMAC-SHA256)	201	401, 403, 422
`DELETE`	`/api/v1/employer/webhooks/{hid}`	Удалить вебхук	200	401, 403, 422

Справочники и публичные данные

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/catalog`	Справочник: направления, специализации, грейды, навыки, дисциплины ФСП	200	
`GET`	`/api/v1/public/stats`	Статистика платформы для главной страницы	200	
`GET`	`/api/v1/public/verify/{token}`	Проверка подлинности PDF-профиля по QR-коду	200	404, 422
`GET`	`/api/v1/vacancies`	Опубликованные вакансии; для кандидата — с оценкой соответствия	200	422
`GET`	`/api/v1/vacancies/{vid}`	Вакансия	200	404, 422
`GET`	`/api/v1/notifications`	Мои уведомления	200	401, 403, 422
`POST`	`/api/v1/notifications/read-all`	Отметить все как прочитанные	200	401, 403, 422
`GET`	`/api/v1/mechanics/bank`	Обзор банка шаблонов-генераторов	200	
`GET`	`/api/v1/mechanics/preview`	Сгенерировать несколько вариантов одного шаблона (без ключей)	200	422
`GET`	`/api/v1/mechanics/simulate`	Симуляция адаптивного теста для «виртуального кандидата»	200	422
`GET`	`/api/v1/validation/report`	Результаты процедуры валидации тестирования и подбора	200	
`POST`	`/api/v1/validation/external-eval`	Оценка подбора на своём размеченном наборе пар «вакансия — кандидат»	200	400, 422, 429
`GET`	`/api/v1/validation/external-eval/example`	Пример размеченного набора пар в формате проверки (из синтетической популяции, разметка по скрытой истине)	200	

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/health`	Проверка работоспособности	200	

ATS-интеграция

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/ats/v1/candidates`	Кандидаты, принявшие приглашения (с контактами) — для импорта в ATS	200	401, 422
`GET`	`/api/v1/ats/v1/vacancies`	Вакансии компании на платформе (с идентификаторами ATS)	200	401, 422
`PUT`	`/api/v1/ats/v1/vacancies/{external_id}`	Создать или обновить вакансию из ATS (специализацию, грейд и навыки распознает разбор текста)	200	401, 422
`GET`	`/api/v1/ats/v1/applications`	Отклики на вакансии компании (с контактами откликнувшихся) — для импорта в ATS	200	401, 422
`PUT`	`/api/v1/ats/v1/applications/{app_id}/status`	Синхронизация статуса отклика из ATS (просмотрен, интервью, оффер, отказ)	200	401, 404, 422
`PUT`	`/api/v1/ats/v1/invitations/{inv_id}/stage`	Синхронизация этапа воронки из ATS	200	401, 422

Администрирование

Метод	Путь	Назначение	Успех	Ошибки
`GET`	`/api/v1/admin/overview`	Сводка платформы и очереди модерации	200	401, 403, 422
`POST`	`/api/v1/admin/recalibrate`	Онлайн-калибровка трудности шаблонов и детекция дрейфа (признак утечки)	200	401, 403, 422

7. ИСПОЛЬЗУЕМЫЕ БИБЛИОТЕКИ И КОМПОНЕНТЫ

Весь код открыт, не скомпилирован и не обфусцирован. Все зависимости — открытое ПО с разрешительными лицензиями (MIT, BSD, Apache-2.0, ISC, PostgreSQL License, OFL для шрифтов).

Backend (Python 3.13)

Библиотека	Версия	Лицензия	Для чего
FastAPI	0.142.2	MIT	HTTP API, OpenAPI/Swagger
Starlette	1.7.0	BSD-3	ASGI-основа FastAPI
Uvicorn (standard)	0.54.0	BSD-3	ASGI-сервер
SQLAlchemy	2.1.3	MIT	ORM, слой хранения
psycopg (binary)	3.3.6	LGPL-3.0 (динамическая линковка)	драйвер PostgreSQL
Pydantic / pydantic-settings	2.13.5 / 2.15.0	MIT	схемы, валидация, конфигурация
email-validator	2.3.0	Unlicense	проверка e-mail
PyJWT (crypto)	2.15.1	MIT	JWT RS256, проверка id_token ФСП ID
cryptography	50.0.2	Apache-2.0 / BSD	RSA-ключи, JWKS
argon2-cffi	25.1.0	MIT	хеширование паролей Argon2id
httpx	0.28.1	BSD-3	OIDC-клиент, реестр ФСП, вебхуки
python-multipart	0.0.32	Apache-2.0	загрузка PDF-резюме
pypdf	6.19.0	BSD-3	извлечение текста из PDF
ReportLab	5.0.1	BSD	генерация PDF-профиля, QR-код
RapidFuzz	3.14.6	MIT	нечёткое сопоставление навыков, автопроверка задач
NumPy	2.5.3	BSD-3	IRT, EAP, калибровка
pytest	9.1.1	MIT	автотесты (dev)
matplotlib	3.11.2	PSF	графики валидации (dev)
sqlite3 (стандартная библиотека)	—	PSF	вычисление эталонных ответов SQL-заданий

Свои реализации (без внешних ML-зависимостей, чтобы укладываться в скромные ресурсы): IRT 3PL, EAP, person-fit lz, адаптивный алгоритм, онлайн-калибровка, TF-IDF по псевдоосновам слов, разбор резюме и потребностей, логистическая модель вероятности принятия приглашения.

Сервис fsp-mock

FastAPI 0.142.2, Uvicorn 0.54.0, PyJWT 2.15.1, cryptography 50.0.2, python-multipart 0.0.32.

Frontend (Node.js 22)

Библиотека	Версия	Лицензия	Для чего
React / React DOM	18.3.1	MIT	UI
React Router	6.30.6	MIT	маршрутизация

Библиотека	Версия	Лицензия	Для чего
TanStack Query	5.104.1	MIT	запросы, кэш
Recharts	3.10.1	MIT	графики (радар компетенций, траектория CAT, валидация)
lucide-react	1.52.0	ISC	иконки
sonner	2.0.8	MIT	уведомления
clsx	2.1.1	MIT	классы
Tailwind CSS	3.4.19	MIT	стили
Vite	5.4.21	MIT	сборка, dev-сервер
TypeScript	5.9.3	Apache-2.0	типизация
PostCSS / Autoprefixer	8.5.28 / 10.6.1	MIT	CSS-обработка

Шрифты — из фирменного стиля ФСП, раздаются с собственного сервера (без внешних CDN, лицензии лежат рядом с файлами): JetBrains Mono (основной шрифт ФСП, OFL 1.1), TDAText (акцентный шрифт ФСП, OFL 1.1), Inter (основной шрифт флагманских проектов ФСП, OFL 1.1). Файлы урезаны до латиницы, кириллицы и нужных символов (`fontTools`) и сохранены в WOFF. В PDF-профиле кандидата — JetBrains Mono и TDAText из `backend/app/assets/fonts`, текст — DejaVu Sans.

Инфраструктура

Компонент	Версия	Лицензия
PostgreSQL	16 (alpine)	PostgreSQL License
nginx	1.27 (alpine)	BSD-2
Mailpit	1.21	MIT
Docker / Docker Compose	24+ / v2	Apache-2.0
Базовые образы	python:3.13-slim, node:22-alpine	—

8. СБОРКА, РАЗВЁРТЫВАНИЕ И ЛОКАЛЬНЫЙ ЗАПУСК

8.1. Docker Compose (рекомендуется)

Требования: Docker 24+ и Docker Compose v2; 2 CPU, 2 ГБ RAM (стек потребляет < 1 ГБ); свободные порты 8080, 8000, 8081, 8025.

```
git clone <репозиторий> draft && cd draft
cp .env.example .env            # при необходимости поменять секреты и адреса
docker compose up --build -d
docker compose ps               # все сервисы — Up / healthy
```

Сервис	Порт	Назначение
frontend (nginx)	8080	веб-приложение, прокси `/api` → backend
backend	8000	API; Swagger — http://localhost:8080/api/docs
fsp-mock	8081	демо ФСП ID (OIDC) и реестр достижений
mailpit	8025	веб-почта демо-стенда (письма подтверждения e-mail)
db	—	PostgreSQL 16, том `pgdata`

Первый старт: `start.sh` в контейнере backend генерирует ключ подписи JWT (том `backend-var`), создаёт схему БД и заполняет синтетическими данными (под advisory-lock PostgreSQL), затем запускает 2 воркера uvicorn.

Сброс данных: `docker compose down -v && docker compose up -d`.

Переменные окружения (основные)

Переменная	По умолчанию	Описание
`DATABASE_URL`	`postgresql+psycopg://draft:draft@db:5432/draft`	строка подключения
`PUBLIC_URL`	`http://localhost:8080`	публичный адрес фронтенда (ссылки в письмах, QR в PDF, OIDC redirect)
`SECRET_KEY`	—	HMAC для подписи PDF-профилей и вебхуков — **заменить**
`JWT_ISSUER`, `ACCESS_TOKEN_TTL_MIN`, `REFRESH_TOKEN_TTL_DAYS`	`…/realms/draft`, 30, 14	параметры токенов
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_TLS`, `MAIL_FROM`	mailpit:1025	почта
`DEMO_MODE`, `DEMO_MAILBOX_URL`	true, http://localhost:8025	демо-подсказки на страницах
`FSP_OIDC_PUBLIC_BASE`, `FSP_OIDC_INTERNAL_BASE`, `FSP_REALM`, `FSP_CLIENT_ID`, `FSP_CLIENT_SECRET`, `FSP_REGISTRY_API`	см. compose	ФСП ID и реестр (п. 5.5)
`TEST_MIN_ITEMS`, `TEST_MAX_ITEMS`, `TEST_SE_TARGET`	10, 16, 0.35	длина и точность адаптивного теста

Переменная	По умолчанию	Описание
`GRADE_CHANGE_COOLDOWN_DAYS`, `RETAKE_SAME_GRADE_COOLDOWN_DAYS`, `MAX_ATTEMPTS_PER_30_DAYS`	90, 14, 4	ограничения пересдач
`TASK_ASSIGN_INTERVAL_DAYS`, `TASK_DUE_DAYS`	7, 7	регулярные задачи
`SEED_ON_START`, `SEED_CANDIDATES`	true, 720	синтетические данные
`WEB_CONCURRENCY`	по числу ядер, до 4	число воркеров uvicorn
`FIELD_ENCRYPTION_KEY`	производный от `SECRET_KEY`	ключ шифрования ФИО и контактов в БД (base64, 32 байта)

Публичный стенд одной командой

Нужен сервер с Docker (Ubuntu 22.04+, от 2 vCPU / 2 ГБ RAM) с открытыми портами 80 и 443. Домен не обязателен.

```
git clone <репозиторий> draft && cd draft
./deploy/deploy.sh                          # адреса вида https://draft.<ip>.sslip.io
BASE_DOMAIN=example.ru ./deploy/deploy.sh   # свой домен: A-записи draft., fsp., mail. → IP сервера
```

Скрипт генерирует `.env` со случайными секретами и поднимает стек с наложением `deploy/docker-compose.prod.yml`: перед сервисами встаёт Caddy (`deploy/Caddyfile`) с автоматическими сертификатами Let's Encrypt, наружу открыты только 80/443, база, backend, мок ФСП ID и почта доступны лишь внутри сети Docker. Публичные адреса платформы и ФСП ID прописываются в `PUBLIC_URL`, `FSP_OIDC_PUBLIC_BASE`, `JWT_ISSUER` и в список разрешённых адресов возврата OIDC. Конфигурация проверена локально на доменах `*.localhost` (внутренний центр сертификации Caddy): HTTPS на всех адресах, редирект HTTP → HTTPS, вход через ФСП ID через публичные адреса, аудит требований ТЗ — 73/73.

Для боевого режима: адрес Keycloak ФСП вместо мока (`FSP_OIDC_PUBLIC_BASE`, `FSP_OIDC_INTERNAL_BASE`), реальный SMTP и `DEMO_MODE=false`, резервное копирование тома `pgdata` (`pg_dump` по расписанию).

Если порт 443 уже занят другим сервисом

Так развёрнут демо-стенд: на сервере уже работает VPN (ocserv) на 443. Порт делится по имени без расшифровки трафика: HAProxy (`deploy/haproxy-sni.cfg`, отдельный контейнер в сети хоста) смотрит на имя в TLS ClientHello — домены стенда уходят в Caddy, всё остальное (другие домены и подключения по IP) — прежнему сервису, перенесённому на внутренний порт. Оба получают реальный IP клиента через PROXY protocol, UDP 443 у VPN не меняется, клиентам VPN ничего перенастраивать не нужно. Порт 80 занимает Caddy; сертификаты других сервисов хоста продлеваются через `certbot --webroot` (Caddy отдаёт файлы проверки из `/var/www/acme`, список доменов — `ACME_WEBROOT_SITES`). На слабом сервере фронтенд собирается заранее (`npm run build`), образ — из готового `dist` (`frontend/Dockerfile.prebuilt`). Стенд работает на домене `sporthackathon.ru` (`APP_DOMAIN`, `FSP_DOMAIN=fsp.…`, `MAIL_DOMAIN=mail.…` в `.env`); дополнительные имена (`www`, прежний технический адрес на sslip.io) перечислены в `APP_REDIRECT_FROM` — Caddy выпускает для них сертификаты и переадресует на основной домен. HAProxy пропускает в стенд сам домен и все его поддомены (`DRAFT_SNI_DOMAIN`).

```
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml -f deploy/docker-compose.sni.yml up -d --build
docker run -d --name sni-proxy --network host --restart always --user root \
  -e DRAFT_SNI_SUFFIX=.46-29-164-196.sslip.io -e DRAFT_SNI_DOMAIN=sporthackathon.ru -e EXISTING_TLS_BACKEND=127.0.0.1:4443 \
  -v /etc/haproxy-sni/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro haproxy:2.8-alpine
```

Демо-видео

`docs/demo/draft-demo.mp4` (4 минуты 20 секунд, 1440×900): сквозные сценарии кандидата и работодателя, механика тестирования, валидация и вход через ФСП ID. Видео записывается автоматически сценарием `docs/demo_video.py` (headless Chrome + DevTools Protocol, подписи на экране), поэтому его можно перезаписать после любых изменений интерфейса:

```
backend/.venv/bin/pip install imageio-ffmpeg && backend/.venv/bin/python docs/demo_video.py
```

8.2. Локальный запуск без Docker (разработка)

Требования: Python 3.12+ (проверено на 3.13), Node.js 20+ (проверено на 22), PostgreSQL 14+.

```
createdb draft
cd backend
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.txt
cat > .env <<'EOF'
DATABASE_URL=postgresql+psycopg://localhost/draft
PUBLIC_URL=http://localhost:5173
EOF
.venv/bin/uvicorn app.main:app --port 8000 --reload
```

```
cd fsp-mock
../backend/.venv/bin/pip install -r requirements.txt
../backend/.venv/bin/uvicorn app.main:app --port 8081
```

```
cd frontend
npm install
npm run dev          # http://localhost:5173, прокси /api → localhost:8000
```

Без SMTP-сервера письма не отправляются, а код подтверждения выводится в лог backend (строка «SMTP недоступен…»). Для локальной почты можно запустить только Mailpit: `docker run -p 1025:1025 -p 8025:8025 axllent/mailpit`.

8.3. Проверки

```
cd backend
createdb draft_test                                  # отдельная БД для тестов
.venv/bin/python -m pytest -q                        # 278 тестов
.venv/bin/python -m app.validation.run_all           # процедура валидации → validation_results/ и docs/04-validation.md
.venv/bin/python -m app.validation.tune_matching     # подстройка весов подбора на отдельных наборах →
app/matching/tuned_params.json
.venv/bin/python -m app.validation.code_exec --docker  # ключи ответов: исполнение кода на 9 языках в Docker-образах
cd ../frontend && npm test                           # 14 тестов компонентов (Vitest)
npm run build                                        # проверка типов и сборка
npx playwright test                                  # 7 браузерных сценариев на запущенном стенде (E2E_BASE_URL)
```

CI (`.github/workflows/ci.yml`) на каждую загрузку: автотесты бэкенда с компиляторами 9 языков, тесты и сборка фронтенда, поиск секретов в истории (gitleaks), запуск всего стенда в Docker, сквозная проверка API и браузерные сценарии.

8.4. Частые проблемы

Симптом	Решение
Не приходит письмо с кодом	открыть Mailpit http://localhost:8025; локально без SMTP — код в логе backend
«ФСП ID временно недоступен»	проверить, что запущен fsp-mock (порт 8081) и совпадают `FSP_OIDC_*`
Ошибка `invalid_state` при входе через ФСП ID	сессия авторизации старше 10 минут — повторить вход
Порт занят	изменить левую часть маппинга портов в `docker-compose.yml`

Симптом	Решение
Нужно сбросить демо-данные	`docker compose down -v` (локально — пересоздать БД)

9. БЕЗОПАСНОСТЬ И 152-ФЗ

9.1. Персональные данные (Федеральный закон № 152-ФЗ)

Требование	Реализация
Согласие на обработку ПДн	обязательный чекбокс при регистрации (сервер отклоняет регистрацию без него)
Отдельное согласие на публикацию профиля	кандидат управляет видимостью в базе; отзыв согласия скрывает профиль из поиска и подборок
Согласие на получение данных из реестра ФСП	фиксируется при связывании ФСП ID, отзыв удаляет полученные достижения
Журнал согласий	таблица `consents`: вид, версия документа, решение, дата, IP, User-Agent; виден кандидату в настройках
Минимизация раскрытия	работодатель до принятия приглашения видит код кандидата вместо имени и не видит контактов; текущий работодатель скрыт; кандидат может скрыть зарплатные ожидания, ФСП, результаты задач
Право на доступ	выгрузка всех своих данных в JSON (`GET /candidate/export`)
Право на удаление	удаление аккаунта со всеми связанными данными (`DELETE /candidate/account`, каскадное удаление)
Аудит	`audit_log`: раскрытие контактов, просмотры профиля работодателем, выгрузки, связывание ФСП ID, удаления
Отзыв доступа к контактам	после принятия приглашения кандидат может закрыть контакты: они исчезают из кабинета, API и ATS-выгрузки работодателя; действие пишется в аудит
Несовершеннолетние (15–17 лет)	регистрация только с согласием законного представителя (запись в журнале согласий); работодатель видит пометку и ограничения ТК РФ
Шифрование в БД	ФИО, телефон и Telegram кандидата хранятся зашифрованными (AES-256-GCM, случайный nonce, имя поля как AAD; ключ `FIELD_ENCRYPTION_KEY` или производный от `SECRET_KEY`) — дамп базы не раскрывает контакты
Хранение	PostgreSQL в контуре оператора; в продакшене — размещение в РФ, шифрование дисков и резервных копий

- Секреты не попадают в репозиторий: `.env`, ключи JWT и тестовые ключи — в `.gitignore`; CI проверяет всю историю репозитория сканером **gitleaks**.

9.2. Аутентификация и сессии

- Регистрация по e-mail с подтверждением: 6-значный код (хранится только SHA-256-хеш) или одноразовая ссылка, срок 24 часа.
- Пароли — **Argon2id** (параметры OWASP: m = 19 МиБ, t = 2), требования к сложности на сервере.
- Access-токен JWT **RS256**, 30 минут, Keycloak-совместимые claims; публичный ключ в JWKS.
- Refresh-токен — случайный 320-бит, в БД хранится хеш; **ротация** при каждом обновлении, отзыв при выходе.
- Защита от перебора: 5 неудачных входов → блокировка на 5 минут; rate limiting чувствительных эндпоинтов (20 запросов/мин с IP).
- Вход через ФСП ID: Authorization Code + **PKCE S256**, проверка `state` (≤ 10 мин), `nonce`, подписи `id_token` по JWKS, `iss` и `aud`; автоматического связывания по совпадению e-mail нет (защита от захвата аккаунта).

9.3. Разграничение прав

- Роли `candidate`, `employer`, `admin` в `realm_access.roles`; зависимости `require_role` на каждом роутере.
- Кабинеты доступны только с подтверждённым e-mail.

- Объектный уровень: каждый запрос к приглашению, отклику, потребности, вакансии, задаче проверяет владельца (чужой объект → 404, без раскрытия факта существования). Проверено автотестами.
- ATS-доступ — отдельные API-ключи (хранится хеш, префикс для поиска), отзыв в один клик.

9.4. Сетевой периметр и OIDC

- На публичном стенде наружу открыты только 80/443 (Caddy, TLS); база, API, мок ФСП ID и почта — во внутренней сети.
- Реальный IP клиента за прокси берётся из `X-Forwarded-For` (`uvicorn --proxy-headers --forwarded-allow-ips`), поэтому лимиты частоты входа и регистрации считаются на пользователя, а не на прокси.
- Демо-стенд ФСП ID, как и Keycloak, принимает только зарегистрированные адреса возврата (`FSP_ALLOWED_REDIRECTS`): код авторизации не может уйти на чужой домен (открытый редирект закрыт, проверяется и подмена префиксом).

9.5. Защита тестирования

Ключи ответов не покидают сервер; время ответа считается по серверным часам; одно активное задание на попытку (защита от повторной отправки), лимиты попыток; сигналы нечестности (уходы со вкладки, вставки, сверхбыстрые ответы, person-fit) — см. п. 2.6.

9.6. Прочее

- Заголовки безопасности: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`.
- CORS — только для адресов фронтенда.
- Валидация всех входных данных Pydantic-схемами (длины, диапазоны, форматы, ИНН); загрузка резюме — только PDF до 5 МБ.
- Подпись PDF-профиля (HMAC) и QR-проверка подлинности; подпись вебхуков HMAC-SHA256.
- Контейнеры запускаются от непривилегированного пользователя.

10. АНТИФРОД, ИНТЕГРАЦИЯ С ATS И РАЗВИТИЕ ПОСЛЕ MVP

10.1. Защита от фиктивных вакансий и недобросовестных работодателей

Реализовано в прототипе

Механизм	Как работает
Индекс доверия компании (0–100)	база 40; +15 корпоративная почта на домене сайта; +10 ИНН прошёл проверку контрольной суммы; +10 заполненный профиль; до +10 за возраст аккаунта; ±10 доля принятых приглашений; −15 за каждую открытую жалобу; −10 за вакансии с флагами риска. Факторы видны работодателю, индекс — кандидату в каждом приглашении
Лимиты приглашений	в сутки: 15 (низкое доверие) / 40 (среднее) / 120 (высокое) — массовая рассылка без репутации невозможна
Флаги риска текста	предоплата и «оплата обучения», финансовые схемы, запрос паспортных и платёжных данных, «доход без опыта», перевод общения в сторонние каналы, слишком короткое описание. Критичные флаги блокируют публикацию вакансии и отправку приглашения (`422 risky_content`), остальные показываются кандидату предупреждением
Реалистичность вилки	сравнение с рыночной медианой категории: «втрое выше рынка» — флаг риска; «заметно ниже рынка» — подсказка работодателю
Жалобы кандидатов	«Пожаловаться» в приглашении: причина + комментарий; жалоба сразу снижает индекс доверия, очередь модерации у администратора
Аналитика отказов	работодатель видит, почему ему отказывают; систематически заниженные вилки видны и платформе

Концепция развития

1. **Верификация юрлица**: ИНН/ОГРН по ЕГРЮЛ (API ФНС / DaData) — статус «действующее», совпадение названия и руководителя; вход через **Госуслуги для юрлиц (ЕСИА)**; подтверждение домена (DNS TXT или письмо на admin@домен).
2. **Модерация с приоритетом риска**: вакансии и первые приглашения новых компаний — в очередь; ML-модель риска (текстовые признаки, вилка, поведение аккаунта) определяет, что проверять вручную.
3. **Поведенческие сигналы**: доля приглашений без последующего контакта, «призрачные» вакансии без откликов и движения, всплески рассылок, повторяющиеся тексты у разных компаний.
4. **Репутация от кандидатов**: короткая оценка после принятия приглашения («условия совпали с обещанными?»), публичный рейтинг ответственности работодателя.
5. **Санкции**: понижение лимитов → скрытие приглашений до модерации → блокировка; апелляция.

10.2. Интеграция с ATS работодателя

Организаторы определили ATS-интеграцию как синхронизацию **вакансий, откликов и статусов** с внутренней системой найма работодателя через API, чтобы платформа встраивалась в существующий процесс без ручного дублирования. В MVP это не обязательно; в «Драфте» сквозной контур уже работает.

Реализовано

Что синхронизируется	Направление	Как
Вакансии	ATS → платформа	`PUT /ats/v1/vacancies/{external_id}` — создание или обновление по идентификатору ATS (повтор не создаёт копию). Специализацию, грейд, навыки и формат, если их нет в запросе, распознаёт тот же разбор текста, что и у потребностей; без уверенно распознанной специализации — ошибка 422, а не угаданная категория. Антифрод-проверка текста — как у ручной публикации. `GET /ats/v1/vacancies` — вакансии с `external_id`

Что синхронизируется	Направление	Как
Отклики	платформа → ATS	`GET /ats/v1/applications?since=…` — отклики с контактами и `external_id` вакансии; вебхук `application.created`
Кандидаты из приглашений	платформа → ATS	`GET /ats/v1/candidates` — принявшие приглашение, с категорией, подтверждёнными навыками и ФСП; вебхук `invitation.accepted` (отзыв контактов кандидатом убирает их из выгрузки)
Статусы откликов	обе стороны	из ATS — `PUT /ats/v1/applications/{id}/status` (кандидат получает уведомление); с платформы — вебхук `application.status_changed`
Этапы найма по приглашению	обе стороны	из ATS — `PUT /ats/v1/invitations/{id}/stage`; с платформы — вебхук `invitation.stage_changed`

- Вебхуки подписаны `X-Draft-Signature: sha256=HMAC(secret, body)`, событие — в `X-Draft-Event`, 3 попытки с экспоненциальной задержкой, журнал доставки. События: `invitation.accepted`, `invitation.declined`, `invitation.stage_changed`, `application.created`, `application.status_changed`, `task.submitted`.
- Управление ключами и вебхуками — в кабинете («Интеграции (ATS)»); ключи хранятся хешем, отзываются в один клик.

Концепция

- Готовые коннекторы к российским ATS (Huntflow, Potok, Talantix, E-Staff, Skillaz) поверх API выше: маппинг их моделей (вакансия ↔ `external_id`, этапы воронки ↔ `stage` / `status`), периодическая сверка на случай потерянного вебхука.
- Экспорт кандидата в стандартизированный JSON (JSON Resume + блок `verification`: категория, θ-перцентиль, подтверждённые навыки, ссылки на подписанные записи реестра ФСП) и PDF-профиль с QR-проверкой.
- OAuth 2.0 client credentials для ATS вместо статических ключей, скоупы `candidates:read`, `invitations:write`.

10.3. Развитие механик

Направление	Что делаем
Тестирование	калибровка на реальных прохождениях (MML), анализ DIF между языковыми треками, **задания с исполнением кода** в песочнице (интеграция с проверяющими системами соревнований ФСП), расширение банка до 500+ семейств, прокторинг для подозрительных попыток, тесты для Lead-ролей (архитектурные кейсы с экспертной оценкой по рубрике)
Подбор	обучение весов ранжирования на исходах приглашений (learning to rank: принято / собеседование / найм), учёт истории отказов кандидата, «похожие кандидаты», уведомление работодателя о новых кандидатах в сохранённой подборке
Кандидатам	персональный план развития по слабым компетенциям, рекомендации соревнований ФСП, «рыночная стоимость» категории
ФСП	событийная синхронизация реестра, отображение командных ролей и вклада, бейджи ФСП в профиле, совместные задачи от партнёров ФСП как регулярные задания

10.4. Дизайнеры: как тестировать объективно

Организаторы допускают дизайнеров в справочнике специализаций, но приоритет — профильные ИТ-специалисты, у которых тестирование объективнее. Поэтому в MVP дизайнеров нет, а механика для них уже понятна и укладывается в тот же движок:

Компетенция	Задание с однозначным ответом (генерируется заново для каждого кандидата)
Доступность	контраст пары цветов по WCAG 2.2 (выдержит ли AA/AAA для текста этого кегля), минимальный размер области нажатия, порядок фокуса

Компетенция	Задание с однозначным ответом (генерируется заново для каждого кандидата)
Сетка и вёрстка	ширина колонки 12-колоночной сетки при заданных полях и гаттерах, кратность отступов шагу 4/8 pt, пересчёт макета под брейкпоинт
Типографика	размер ступени модульной шкалы, межстрочный интервал по ритму, длина строки в знаках
Цвет	перевод HEX ↔ RGB ↔ HSL, подбор оттенка с заданным контрастом к фону
UX-закономерности	закон Фиттса и Хика (какой вариант быстрее), эвристики Нильсена — какая нарушена на показанном экране
Продуктовые метрики	конверсия воронки, значимость A/B-теста макетов

Неформализуемая часть — композиция, вкус, работа с исследованием — проверяется **регулярной задачей с рубрикой**: работодатель или эксперт ФСП оценивает решение по заранее заданным пунктам, а автоматическая предварительная оценка и проверка на списывание работают так же, как у разработчиков (п. 2.8). Портфолио — самоописание, оно на грейд не влияет.

10.5. Технический долг и масштабирование

- Миграции схемы через Alembic (сейчас схема создаётся из метаданных SQLAlchemy при старте — идемпотентно).
- Очередь задач (Redis + RQ/Celery или Kafka) для вебхуков, писем, пересчёта подборок и синхронизации ФСП.
- Кэш справочников и банка в Redis; read-реплики PostgreSQL для поиска; выделение сервисов тестирования и подбора при росте нагрузки.
- Наблюдаемость: метрики Prometheus (RPS, латентность подборки, экспозиция семейств), трассировка, алерты по SLO.

11. СООТВЕТСТВИЕ РЕШЕНИЯМ СЕССИИ ВОПРОСОВ И ОТВЕТОВ

На сессии вопросов и ответов постановщики (ментор, юрисконсульт ФСП, эксперт по веб-платформам, продуктовый специалист ФСП) уточнили требования. Ниже — каждое решение и то, как оно реализовано в «Драфте».

11.1. Тест, грейд, категория

Решение на сессии	Как в «Драфте»	Где
Грейд — только по первичному тесту категории, не по вакансии и не по достижениям ФСП	Грейд меняет только завершённая попытка теста по специализации. ФСП, резюме и вакансия на грейд не влияют: итоговая θ оценивается с популяционным априорным распределением, ФСП в тест не входит	`testing/service.py: finish_attempt`, п. 2.4
Достижения ФСП — ранжирование внутри категории	Компонента F силы профиля и формулы подбора; на категорию не влияет	п. 3.2, 5.3
Неподтверждённый грейд — показывать со статусом и опускать, не скрывать	Кандидат с заявленным, но не подтверждённым грейдом виден в категории заявленного грейда со статусом «грейд не подтверждён тестом», **после всех подтверждённых**; L = 0, оценка × 0.6; фильтр «только подтверждённые»; отдельный счётчик на карте категорий	п. 3.3, `matching/service.py`
Кандидат сам выбирает грейд; подсказка по резюме допустима	Грейд выбирается в опросе; рядом подсказка по стажу (в том числе распознанному из резюме) — только ориентир	экран «Категория и тестирование»
Повышение — лучше отдельным тестом, не автопрыжок	После уверенного прохождения кандидату предлагается **отдельный тест на грейд выше** — сразу, без ожидания 90 дней и вне лимита попыток. Автоматического повышения нет	п. 2.4, `POST /attempts/{id}/upgrade`
Нельзя скакать по грейдам; провал и уход ниже — без сжигания попытки	Смена грейда — раз в 90 дней и на одну ступень; повтор того же уровня — через 14 дней. Шаг вниз после неудачи **не расходует попытку** и не запускает 90-дневную блокировку	п. 2.4, `tests/test_grade_rules.py`
Одинаковые задачи не должны расходиться между кандидатами	226 семейств-генераторов: у каждого кандидата свои данные и ответ; **сервер не выдаёт уже показанный кому-либо вариант**; контроль экспозиции. Атака утечкой: из утечки 0% заданий, Junior проходит Middle в 5% (без утечки — 7%, у статичного банка — 97%)	п. 2.1, 2.5, 4
Задания нейросетью допустимы, но нужны уникальность и мало ошибок	LLM не используются: ответ вычисляет генератор, и все 918 проверенных вариантов с кодом **исполнены настоящими компиляторами 9 языков и SQLite — расхождений нет**	п. 2.1, `validation/code_exec.py`
Песочница, прокторинг, антиплагиат, защита от LLM — плюс	Таймер на каждое задание, учёт уходов со вкладки и вставок, запрет копирования кода, person-fit (индекс lz); при сигналах грейд подтверждается **контрольной формой** из свежих заданий; уникальные варианты (готовый ответ «из интернета» не подходит). Ограничение живой подсказки без прокторинга описано честно	п. 2.6

Решение на сессии	Как в «Драфте»	Где
Задания от работодателя — плюс; эталон проверяет система, открытый ответ — работодатель	Регулярные задачи: автоматическая предварительная оценка по скрытой рубрике и итоговая оценка работодателя	п. 2.8
Несколько категорий у кандидата — плюс	Основная категория и **до двух дополнительных**, грейд в каждой — только тестом; работодатель находит кандидата во всех категориях. Справочник — 8 специализаций в 5 направлениях	п. 3.2

11.2. ФСП и ФСП ID

Решение на сессии	Как в «Драфте»	Где
Подтверждённый профиль — данные из ФСП ID, а не самозаявленный опыт	Результаты приходят из реестра по ФСП ID, подписаны; модерация не нужна	п. 5
Реестр правдоподобный: места, разряды, дисциплины, командные результаты	Мок-реестр с дисциплинами, уровнями, местами, командами, ролями, стеком решения, разрядом	п. 5.3
Keycloak не поднимать, вход имитировать; альтернатива — регистрация по e-mail	`fsp-mock` — OIDC с URL и токенами Keycloak (Authorization Code + PKCE, JWKS); вход и регистрация по e-mail тоже есть	п. 5.1–5.2
Все соревнования равно важны, рейтинг турниров не нужен	Уровень соревнования **в оценку не входит**, рейтинга дисциплин нет. По идее из письменных ответов учитывается только актуальность дисциплины для специализации (п. 11.6)	п. 5.3
Учитывать число соревнований и результативность, чтобы профиль нельзя было набить онлайн-явками	Каждый призовой и финальный результат добавляет вклад; участия без результата — с потолком 0.6, то есть меньше одного призового места	`fsp/scoring.py`, п. 5.3
Срок давности — по желанию	Вклад результата убывает вдвое за 2 года	п. 5.3
Корректная работа без истории ФСП	F = 0: ни штрафа, ни бонуса, кандидат в выдаче; пометка для работодателя	п. 5.4

11.3. Подбор и выход на контакт

Решение на сессии	Как в «Драфте»	Где
Минимум: категоризация, объяснимый подбор, адресное приглашение; оригинальность — плюс	Потребность текстом → рекомендованные категории → подборка с «почему в подборке» → приглашение с вилкой. Дополнительно: вероятность принятия, уточнение без потери подборки, тест под вакансию, вакансии и отклики	п. 3
Импорт чужих вакансий не нужен	Не делается: только вакансии и потребности работодателей платформы	—
Контакты открываются пригласившему после принятия; отзыв доступа — плюс	После принятия контакты видит только пригласивший работодатель. Кандидат может **закрыть доступ** в любой момент: контакты исчезают из кабинета, API и ATS-выгрузки; действие пишется в аудит	`POST /candidate/invitations/{id}/contacts`
Чат на платформе — плюс	**Переписка внутри принятого приглашения**: без обмена контактами, с отметкой о прочтении и уведомлениями; продолжает работать и после отзыва контактов	`GET/POST …/invitations/{id}/messages`

Решение на сессии	Как в «Драфте»	Где
Зарплата: gross или net — подписать	Все суммы — в месяц, **до вычета НДФЛ (gross)**; подпись у каждой суммы в интерфейсе, письмах и API. Вилку «на руки» из текста вакансии разбор переводит в gross	п. 3.2
Проверка компании — добровольная метка доверия, не блокирует сценарий	Индекс доверия (ИНН, домен почты, жалобы, полнота профиля) виден кандидату; влияет только на суточный лимит приглашений	п. 1.1, `services/trust.py`

11.4. Пользователи и право

Решение на сессии	Как в «Драфте»	Где
Правовой минимум — 152-ФЗ	Отдельные согласия с версией и журналом, отзыв, выгрузка и удаление данных, аудит раскрытия контактов	п. 9
С 15 лет — только лёгкий труд и согласие родителей на ПДн (плюс)	При регистрации кандидат указывает возраст: 18+ или 15–17. Для 15–17 обязательно согласие законного представителя (пишется в журнал согласий). Работодатель видит пометку «15–17 лет · лёгкий труд» и напоминание об ограничениях ТК РФ	экран регистрации, карточка кандидата

11.5. Проверка жюри и сдача

Решение на сессии	Как в «Драфте»	Где
Жюри само проходит тесты и проверяет воспроизводимость и разделяющую силу; экспресс-тест на 3–5 минут допустим	Аккаунты `expert1…5@draft.demo` с **режимом эксперта**: «ответить верно / неверно» и «пройти как профиль уровня θ» (стажёр … senior, в том числе на пороге) — за минуту; тот же профиль в другом аккаунте даёт другие задания и тот же грейд. Можно пройти тест и самому	README, страница теста
Свой набор пар «вакансия — кандидат» и процедура валидации	Синтетическая популяция со скрытой истиной, 5 независимых наборов; инструмент проверки на любом своём наборе пар — в интерфейсе и из консоли	п. 4
Стенд без GPU, ориентир ≈ 8 CPU и 32–64 ГБ	Стек занимает ≈ 1 ГБ RAM под нагрузкой; GPU и внешние API не нужны	п. 1.5
Стабильная работа при десятках одновременных пользователей	50 одновременных пользователей: 432 запроса/с, p95 ≤ 0,30 с, 0 ошибок (20 пользователей: p95 ≤ 0,13 с)	п. 1.5
LLM можно, но описать, какие и зачем	LLM не используются (п. 1.5)	п. 1.5
Обязательный перечень документации	Архитектура (п. 1), механика тестирования (п. 2), подбор и категоризация (п. 3), валидация (п. 4), интеграция с ФСП и ФСП ID (п. 5), API (п. 6, `openapi.json`), библиотеки с версиями (п. 7), сборка и запуск (п. 8)	`docs/`
Упавший контейнер можно переподнять, идею после дедлайна не менять	Все сервисы с `restart: unless-stopped`, инициализация БД идемпотентна. История изменений — в репозитории	`docker-compose.yml`

11.6. Письменные ответы организаторов

После сессии организаторы письменно ответили на вопросы команд трека. Ниже — каждый ответ и как он отражён в «Драфте».

Вопрос → ответ организаторов	Как в «Драфте»	Где
ФСП — буст внутри подтверждённого грейда, не замена теста? → **Да.** Все дисциплины одинаково приоритетны; можно учитывать актуальность направления (ИБ-специалист с соревнованием по ИБ релевантнее, чем с робототехникой) — на усмотрение команды	ФСП двигает кандидата только внутри категории, грейд — только тестом. **Реализована идея актуальности**: вклад результата × 1.0 (профильная дисциплина), 0.8 (смежная), 0.6 (непрофильная); если стек решения совпал с навыками запроса — профильная. Рейтинга дисциплин нет, любой результат засчитывается; в объяснении выдачи написано, профильная ли дисциплина	п. 5.3, `fsp/scoring.py`, `catalog.discipline_fit`
Регулярные задания: адресно или челлендж? Кто оценивает? → **Адресно** (челлендж не запрещён), способ оценки — на усмотрение команды	Адресно: раз в 7 дней задача работодателя категории, с распределением нагрузки между задачами. Оценка: автопроверка по скрытой рубрике → **проверка на списывание** (почти совпадающие ответы на ту же задачу помечаются) → итоговая оценка работодателя 1–5, байесовское среднее	п. 2.8
Питч, видео вместо стенда, срок стенда → **10 минут; видео допустимо; стенд держать на время оценки**	Демо-видео записывается одной командой по актуальному интерфейсу; стенд разворачивается одной командой и переподнимается без потери данных	`docs/demo_video.py`, п. 8
Стабильность грейда при повторном прохождении → **процедуру предлагает команда: test–retest и разделяющая сила заданий, симуляция на синтетике допустима**	1200 виртуальных кандидатов проходят тест дважды на разных формах: корреляция θ 0.88, взвешенная к 0.85, грейды двух прохождений расходятся не больше чем на ступень — 100%; точность грейда относительно истины — 81%, вдали от порога — 90%. Разделяющая сила: AUC соседних грейдов 0.92–0.94, медианная точечно-бисериальная корреляция 0.53. Жюри повторяет это вручную в режиме эксперта	п. 4.2, README
Грейд — стаж или навыки? → **Уровень навыков, подтверждённый тестом**; стаж — атрибут профиля	Грейд присваивает только тест; стаж хранится в профиле и служит подсказкой при выборе грейда	п. 3.2
Несколько категорий у кандидата? → **Не обязательно, будет плюсом**	Основная + до двух дополнительных категорий, у каждой свой грейд, тест, блокировка и паузы; дополнительную можно сделать основной, чтобы пройти по ней тест; работодатель находит кандидата во всех категориях	п. 3.2, `services/categories.py`
Облачные LLM? → **Допустимы, в том числе зарубежные; лучше открытые модели; риски 152-ФЗ — на команде**	LLM не используются: персональные данные не покидают платформу. Если понадобятся — открытая модель у оператора платформы и только обезличенный профиль	п. 1.5
Что видит работодатель до принятия? → **ФИО и места работы можно, закрыты только контакты**	Контакты закрыты до принятия. Имя и текущее место работы кандидат открывает сам: по умолчанию — код кандидата и опыт без текущей компании (слепой отбор, минимизация ПДн)	п. 3.6
Тест единый под категорию или под вакансию? → **Единый под категорию, но не одинаковый пул для всех; задания под вакансию — необязательный блок регулярных заданий**	Тест по категории; у каждого кандидата свои задания (226 семейств-генераторов, адаптивный отбор, сервер не выдаёт уже показанный вариант). Задачи под вакансию — блок регулярных заданий работодателя	п. 2.1, 2.5, 2.8
Ограничения по ресурсам? → **Нет, расход должен быть оправдан**	≈ 1 ГБ RAM под нагрузкой, без GPU и внешних API	п. 1.5

Вопрос → ответ организаторов	Как в «Драфте»	Где
Скорость или качество подбора? → **Качество (35% против 7%)**	Веса подобраны на отдельных наборах, качество измерено на независимых (nDCG@10 0.79, P@5 0.85); при этом 50 одновременных пользователей — p95 ≤ 0,30 с	п. 3.3, 4.3
Категория — «специализация + грейд» или детальнее? → **Достаточно специализации и грейда, детальнее — плюс**	Категория = специализация × грейд; дополнительно учитываются трек (язык, фреймворк), стек, роль в команде, отрасль и формат работы	п. 3.2
Насколько жёстко подтверждать грейд, нужна ли проверка на списывание? → **Грейд выбирает кандидат, без теста — статус «не подтверждён»; официально — только по тесту. Проверка на списывание не обязательна, но большой плюс**	Неподтверждённый грейд виден со статусом и ниже подтверждённых. Против списывания: уникальные варианты, таймер на задание, уходы со вкладки и вставки, person-fit, контрольная форма при сигналах; проверка на списывание в регулярных заданиях	п. 2.5, 2.6, 2.8
Что такое ATS-интеграция? → **Синхронизация вакансий, откликов и статусов с системой найма работодателя через API; в MVP — концепция (7%)**	Сквозной контур работает: вакансии из ATS по `external_id` с разбором текста, отклики и принявшие приглашение — в ATS, статусы и этапы — в обе стороны (API по ключу + подписанные вебхуки)	п. 10.2
Источники данных? → **Собственные (синтетика или открытые источники); парсинг — только для тестовых данных, не как продуктовая механика**	Синтетическая популяция со скрытой истиной и мок-реестр ФСП; чужие вакансии не собираются и не показываются	п. 4.1, 5
Минимальный охват? → **Двух специализаций с двумя грейдами достаточно, важен сквозной путь от теста до приглашения**	8 специализаций × 4 грейда; сквозной путь «опрос → тест → категория → подборка → приглашение → контакты» проверен автотестами и браузерными сценариями	README, `tests/`, `frontend/e2e`
Дизайнеры? → **Могут быть в справочнике, приоритет — ИТ-роли; идея объективной проверки дизайнеров — плюс**	В MVP — ИТ-роли. Описана механика объективной проверки дизайнеров тем же движком: контраст WCAG, сетка и отступы, типографическая шкала, цветовые модели, законы Фиттса и Хика, эвристики Нильсена; неформализуемое — регулярной задачей с рубрикой	п. 10.4